

DaTra: Data Transformation Semantics as a Symmetric Monoidal Category on Atlas Presheaves

@qualiascript

ChatGPT

1 Dominions

Definition 1.1 (The Category of Dominions). The **Category of Dominions**, denoted $\mathbf{Dom}$, has as its objects sets whose cardinality is at most ω_0 , equivalently sets admitting an injection into ω_0 , and as its morphisms set-theoretic functions.

Definition 1.2 (The Domanial Embedding Functor). The **Domanial Embedding Functor**, denoted $\mathbf{Emb} : \mathbf{Dom} \rightarrow \mathbf{Set}$, sends each dominion to its underlying set.

2 Domanial Insertions

Definition 2.1 (The Category of Domanial Insertions). The **Category of Domanial Insertions**, denoted $\mathbf{DomIns}$, has dominions as objects and monomorphisms in $\mathbf{Dom}$ as morphisms.

Definition 2.2 (The Category of Codomanial Insertions). The **Category of Codomanial Insertions**, denoted $\mathbf{CoDomIns}$, is the opposite category $\mathbf{DomIns}^{\text{op}}$.

3 Chains

Definition 3.1 (Chain). A **Chain** is a totally ordered thin category, indexed skeletally by an ordinal strictly smaller than $\omega_0^{\omega_0}$.

Definition 3.2 (The Spine). The **spine** is the chain with ω_0 objects.

4 Consolidations

Definition 4.1 (The Category of Consolidations). The **Category of Consolidations**, denoted $\mathbf{Con}$, has chains as objects and point-surjective functors between their thin categories as morphisms.

Definition 4.2 (The Category of Coconsolidations). The **Category of Coconsolidations**, denoted $\mathbf{CoCon}$, is the opposite category $\mathbf{Con}^{\text{op}}$.

5 Transportation and Folios

Definition 5.1 (The Transportation Functor). The **Transportation Functor**, denoted $\mathbf{Tra} : \mathbf{Con} \rightarrow \mathbf{Set}$, sends a chain to its object set and a consolidation to its underlying set-theoretic surjection.

Definition 5.2 (Folio). A **folio** is a functor $F : S \rightarrow \mathbf{CoCon}$, where S is the spine, such that $F(0)$ is the singleton chain.

6 Paginations

Definition 6.1 (The Category of Paginations). The **Category of Paginations**, denoted $\mathbf{Pag}$, has as objects pairs (H, E) where $H = \mathbf{Tra} \circ F^{\text{op}}$ for a folio F , and E is the category of elements of H . For $W : \mathbf{Pag}$, write W_H for the first inclusion and W_E for the second inclusion. A morphism $T : X \rightarrow Y$ in $\mathbf{Pag}$ is a functor $T : X_E \rightarrow Y_E$. Thus, for every morphism $f : x \rightarrow y$ of X_E , the following square commutes:

$$\begin{array}{ccc} x = (m, i) & \xrightarrow{f} & (n, X_H(f)(i)) \\ T \downarrow & & \downarrow T \\ (m', i') & \xrightarrow{T(f)} & (n', Y_H(T(f))(i')). \end{array}$$

7 Atlases

Definition 7.1 (The Category of Atlases). The **Category of Atlases**, denoted $\mathbf{Atl}$, has as objects pairs (P, G) , where $P : \mathbf{Pag}$ and $G : P_E \rightarrow \mathbf{DomIns}$ is a functor, equivalently a presheaf $G : P_E^{\text{op}} \rightarrow \mathbf{CoDomIns}$. For $P_H : C \rightarrow \mathbf{Set}$, $m \in |C|$, and distinct elements $k, k' \in |P_H(m)|$, the pullback in G of the arrows induced by $(m \rightarrow 0)(k)$ and $(m \rightarrow 0)(k')$ is empty.

For every $n \in \mathbb{N}$, the **n th atlas page** of X is the set $X_H(n)$ on its full spine. An element $k \in X_H(n)$ is a **page cell**, written (n, k) , and the dominion carried by that cell is $X_G(n, k)$. Thus a page is the indexed collection of cells at one spine position.

There is an integer w such that, for every $w' > w$, $P_H(w' \rightarrow w) = \text{Id}$, and, for every $k \in |P_H(w)|$, the corresponding arrow $G((w' \rightarrow w)(k))$ is the identity. The least such integer is the **cardinality of the atlas**, denoted $|A|$.

For $X : \mathbf{Atl}$, write X_P and X_G for the two components and put $X_H = (X_P)_H$ and $X_E = (X_P)_E$. A morphism $T : X \rightarrow Y$ is a pair (T_P, T_A) , where $T_P : X_P \rightarrow Y_P$ is a pagination morphism and $T_A : X_G \Rightarrow Y_G \circ T_E$ is a natural transformation. Thus, for every $f : x \rightarrow y$ in X_E , the following square commutes:

$$\begin{array}{ccc} X_G(x) & \xrightarrow{X_G(f)} & X_G(y) \\ (T_A)_x \downarrow & & \downarrow (T_A)_y \\ Y_G(T_E(x)) & \xrightarrow{Y_G(T_E(f))} & Y_G(T_E(y)). \end{array}$$

Finally, if $x = |X|$, $x' > x$, and $r \in |X_H(|X| - 1)|$, then $T_P(x', r) = T_P(x, r)$ and $(T_A)_{(x', r)} = (T_A)_{(x, r)}$.

Definition 7.2 (Cardinality of an Atlas). The **cardinality** $|A|$ of an atlas is the least positive integer after which its spine repeats the final genuine page $|A| - 1$.

Definition 7.3 (Extent of an Atlas). The **extent** of A is $\text{Ex}(A) = A_G(0, 0)$. Since objects of $\mathbf{DomIns}$ are dominions, $\text{Ex}(A) : \mathbf{Dom}$ for every $A : \mathbf{Atl}$.

Definition 7.4 (Territory of an Atlas). Let $M = A_H(|A| - 1)$. The **territory** is the indexed family $\text{Ter}(A) : M \rightarrow \mathbf{Dom}$ given by $\text{Ter}(A)(k) = A_G(|A| - 1, k)$.

Definition 7.5 (n th Region of an Atlas). The **n th region** of A is $\text{Ter}(A)(n)$, with indexing beginning at 0.

8 Transposals and Traversals

Definition 8.1 (The Category of Atlas Transposals). The **Category of Atlas Transposals**, denoted AtlTrap , is the wide subcategory of Atl whose morphisms $F : X \rightarrow Y$ have point-injective object maps F_E .

Definition 8.2 (The Category of Atlas Traversals). The **Category of Atlas Traversals**, denoted AtlTrav , is the wide subcategory of AtlTrap whose morphisms satisfy the following conditions. For $F : X \rightarrow Y$, let $x = (m, i)$ and $y = (m', j)$, put $p = \min(m, m')$, write $F_E(x) = (n, i')$ and $F_E(y) = (n', j')$, and put $p' = \min(n, n')$. If

$$X_H(m \rightarrow p)(i) < X_H(m' \rightarrow p)(j),$$

then

$$Y_H(n \rightarrow p')(i') < Y_H(n' \rightarrow p')(j').$$

Furthermore, if $q \in |X|$ and $t \in X_G(q)$ is in the image of a final region—that is, there exist $k \in |\text{Ter}(X)|$ and $l \in \text{Ter}(X)(k)$ whose image under $X_G(|X| - 1 \rightarrow q)(k)$ is t —then this property is preserved by F .

Definition 8.3 (The Category of Stable Atlas Traversals). The **Category of Stable Atlas Traversals**, denoted StaAtlTrav , is the wide subcategory of AtlTrav whose morphisms $F : X \rightarrow Y$ satisfy $F_E(0, 0) = (0, 0)$.

9 Maps and Cartography

Definition 9.1 (The Category of Atlas Maps). The **Category of Atlas Maps**, denoted AtlMap , is the full subcategory of atlases for which every element of the extent is in the image of a final region.

Definition 9.2 (The Atlas Map Inclusion Functor). The **Atlas Map Inclusion Functor** $\text{AtlMapInc} : \text{AtlMap} \rightarrow \text{Atl}$ is the canonical inclusion.

Definition 9.3 (The Category of Atlas Traversal Maps). The **Category of Atlas Traversal Maps**, denoted AtlTravMap , is the full subcategory of AtlTrav whose objects are Atlas Maps.

Definition 9.4 (The Atlas Traversal Map Inclusion Functor). The canonical inclusion is denoted $\text{AtlTravMapInc} : \text{AtlTravMap} \rightarrow \text{AtlTrav}$.

Lemma 9.5 (Cartography Lemma). *The inclusion AtlTravMapInc has a right adjoint, denoted the **Charting Functor** $\text{Chr} : \text{AtlTrav} \rightarrow \text{AtlTravMap}$.*

Proof sketch. On an object $X : \text{AtlTrav}$, the functor Chr sends X to the universal arrow from AtlTravMapInc to X . Write $X' = \text{AtlTravMapInc}(\text{Chr}(X))$, together with $H : X' \rightarrow X$, with X' terminal among objects having this property. The solution is the DaTra map satisfying $\text{Ter}(X') \cong \text{Ter}(X)$. It is unique because AtlTrav preserves orders and terminal because H_E is faithful.

10 Coalitions

Definition 10.1 (The Coalizing Functor). The **Coalizing Functor**, denoted

$$\text{Coa} : \text{StaAtlTrav} \longrightarrow \text{DomIns},$$

is the extent of the charted atlas: $\text{Coa} = \text{Ex} \circ \text{Chr}$.

Definition 10.2 (The Coalition of an Atlas). The **coalition** of an atlas X is the extent of its chart, regarding X as an object of the wide category of stable traversals.

Definition 10.3 (The Domanial Inclusion Functor). The **Domanial Inclusion Functor** $\text{DomInc} : \text{DomIns} \rightarrow \text{StaAtlTrav}$ sends a dominion to the atlas of cardinality 1 whose coalition is that dominion. It is left adjoint to Coa : for $X : \text{DomIns}$ and $Y : \text{StaAtlTrav}$, naturally in X and Y ,

$$\text{Hom}_{\text{StaAtlTrav}}(\text{DomInc}(X), Y) \cong \text{Hom}_{\text{DomIns}}(X, \text{Coa}(Y)),$$

and $\text{Coa}(\text{DomInc}(X)) \cong X$.

11 Data Transformations

Definition 11.1 (The Category of Data Transformations). The **Category of Data Transformations**, also called the category of **Data Transformation Sets** or **DaTra Sets**, is the presheaf category

$$\text{DaTra} = [\text{Atl}^{\text{op}}, \text{Set}].$$

As a presheaf category, it is a topos.

Definition 11.2 (Navigation). A **navigation** of $D : \text{DaTra}$ is a monomorphism $\text{Nav} : \text{Yo}(A) \rightarrow D$ for some atlas A .

Definition 11.3 (Expedition). An **expedition** is a navigation represented by an Atlas Map.

Definition 11.4 (DaTra Restriction). For a wide atlas category W , its **DaTra restriction** is the presheaf category $[W^{\text{op}}, \text{Set}]$. Equivalently, a DaTra set is restricted by retaining exactly its actions along arrows of W . When a horizontal operation is used, its indexing category is first represented by the Atlas Federation of Section 12; this only makes the structural retagging explicit.

Definition 11.5 (The Category of Data Transposals). The **Category of Data Transposals**, denoted DaTrap , is the presheaf category $[\text{AtlTrap}^{\text{op}}, \text{Set}]$.

Definition 11.6 (The Category of Data Traversals). The **Category of Data Traversals**, denoted DaTrav , is the presheaf category $[\text{AtlTrav}^{\text{op}}, \text{Set}]$.

Definition 11.7 (Data Transformation Maps). The **Category of Data Transformation Maps**, denoted DaTraMap , is the full subcategory of DaTra Sets all of whose navigations are expeditions.

12 Atlas Operations

Definition 12.1 (Atlas Merge). The **Atlas Merge** is the object operation

$$\text{AtlMerge} : \text{Ob}(\text{Atl}) \times \text{Ob}(\text{Atl}) \longrightarrow \text{Ob}(\text{Atl}).$$

For atlases X and Y , the extent of $\text{AtlMerge}(X, Y)$ is the disjoint union of their extents. Both atlases are evaluated on their full spines: page $n + 1$ of the merge is the tagged, componentwise combination of page n of X and page n of Y . The stored cardinalities merely record cutoffs after which the corresponding page data are constant; Atlas Merge does not collapse the spine to either finite presentation. Its page 0 is a new singleton page carrying the merged extent, and the two positive-page summands remain distinguished.

Definition 12.2 (Atlas Federation). An **Atlas Federation** is a countable tagged family of atlas objects. The category of Atlas Federations is denoted

$$\text{AtlFed}.$$

A morphism consists of a map of tags and, at each source tag, a stable atlas traversal to its selected target tag. Federation merge is disjoint tagged union of these families. This presentation retains the data of every input while making the empty merge, reassociation, and tag swapping literal.

Definition 12.3 (The Empty Atlas). The **Empty Atlas** is $\text{Atll} = \text{DomInc}(0)$. It is also the distinguished empty atlas used by horizontal sum on full atlas spines.

Definition 12.4 (The Atlas Horizontal Sum Bifunctor). The **Atlas Horizontal Sum Bifunctor**, denoted $\text{AtlHorSum} : \text{AtlFed} \times \text{AtlFed} \rightarrow \text{AtlFed}$, has object action

$$\text{AtlHorSum}(X, X') = \text{AtlMerge}(X, X').$$

Here AtlFed is the category of Atlas Federations; its empty federation represents Atll . Passing to a federation changes no atlas data and asserts no coproduct universal property. Given stable traversals $F : X \rightarrow Y$ and $F' : X' \rightarrow Y'$, horizontal sum preserves the left and right tags and applies F and F' componentwise. Stability fixes the common origin, so this arrow action is again a stable atlas traversal. Thus atlas merge retains the data, while horizontal sum specifies how each tagged side may be used.

Lemma 12.5 (Horizontal Lemma). *The Atlas Horizontal Sum Bifunctor AtlHorSum exists.*

Proof. Identity arrows act identically on every tag, composition holds componentwise, and every component arrow is required to lie in StaAtlTrav . The federation structure supplies the associator, unitors, and braider by reassociation, deletion of the empty tag family, and tag swapping, respectively.

Definition 12.6 (The Atlas Braider). The **Atlas Braider** $\text{AtlBrd}_{F, F'} : \text{AtlHorSum}(F, F') \rightarrow \text{AtlHorSum}(F', F)$ swaps the left and right component tags. Hence $\text{AtlBrd}_{F, F'} \circ \text{AtlBrd}_{F', F} = \text{Id}$.

Definition 12.7 (The Atlas Associator). The **Atlas Associator**, denoted

$$\text{AtlAso}_{F, F', F''} : \text{AtlHorSum}(\text{AtlHorSum}(F, F'), F'') \longrightarrow \text{AtlHorSum}(F, \text{AtlHorSum}(F', F'')),$$

flattens the nested federation and retags its three sides using the canonical reassociation $((x + y) + z) \leftrightarrow (x + (y + z))$. Its data component is the matching reassociation of the three extent summands.

Definition 12.8 (The Atlas Unitors). The **Atlas Left Unitor** $\text{Lu} : \text{AtlHorSum}(\text{Atll}, F) \rightarrow F$ and the **Atlas Right Unitor** $\text{Ru} : \text{AtlHorSum}(F, \text{Atll}) \rightarrow F$ remove the empty tag family. No nonempty atlas page or datum is removed.

13 Stable Data Transformations

Definition 13.1 (Stable Data Transformations). The **Category of Stable Data Transformations**, denoted StaDaTra , is the presheaf category on Atlas Federations whose component arrows lie in StaAtlTrav . Thus stability is part of the indexing category, rather than an additional condition imposed after horizontal composition.

Definition 13.2 (The Empty Map). The **Empty Map**, denoted I , is the Day unit represented by the empty Atlas Federation:

$$I = \text{Yo}(\text{Atll}).$$

Definition 13.3 (The Horizontal Sum Bifunctor). The **Horizontal Sum Bifunctor**, denoted $\text{HorSum} : \text{StaDaTra} \times \text{StaDaTra} \rightarrow \text{StaDaTra}$, or in infix notation by $+_{<} : \text{StaDaTra} \times \text{StaDaTra} \rightarrow \text{StaDaTra}$, is the Day convolution extension of stable atlas horizontal sum. Because the indexing morphisms are arrows of StaAtlTrav , its result and its arrow action land in StaDaTra by construction.

Definition 13.4 (The Stable Data Transformation Braider). The **Stable Data Transformation Braider** $\text{Brd}_{F,F'} : F +_{<} F' \rightarrow F' +_{<} F$ is the Day convolution extension of AtlBrd .

Definition 13.5 (The Stable Data Transformation Associator). The **Stable Data Transformation Associator**

$$\text{Asoc}_{F,F',F''} : (F +_{<} F') +_{<} F'' \longrightarrow F +_{<} (F' +_{<} F'')$$

is the Day convolution extension of AtlAsoc .

Definition 13.6 (The Stable Data Transformation Unitors). The **Stable Data Transformation Left Unitor** $\text{Lu} : I +_{<} F \rightarrow F$ and the **Stable Data Transformation Right Unitor** $\text{Ru} : F +_{<} I \rightarrow F$ are the Day convolution extensions of AtlLu and AtlRu .

Definition 13.7 (The Stable Data Transformations Monoidal Category). The **Stable Data Transformations Monoidal Category**, denoted StaDaTraMon , is

$$\text{StaDaTraMon} = (\text{StaDaTra}, +_{<}, I).$$

Its tensor is Day convolution on the Atlas Federation. The braider, associator, and unitors are induced by retagging that form, so all coherence maps remain within stable data transformations.

A Lean Formalization

The following is an ASCII rendering of the complete Lean source corresponding to the mathematical development above. It replaces Lean’s conventional Unicode surface notation solely for portable typesetting; the checked source file remains `datra.lean`.

```
1  import Mathlib
2
3  namespace Datra
4
5  open CategoryTheory CategoryTheory.Functor CategoryTheory.Limits Opposite
6  open scoped Ordinal
7
8  universe u
9
10 noncomputable section
11
12 /-- A dominion is a set equipped with a certificate of countability. -/
13 structure Dominion where
14   Carrier : Type
15   rank : Function.Embedding Carrier Nat
16
17 abbrev Dom := Dominion
18
19 instance : CoeSort Dominion Type where coe X := X.Carrier
20
21 instance : Category Dominion where
22   Hom X Y := X -> Y
23   id _ := id
24   comp f g := g o f
25   id_comp _ := rfl
26   comp_id _ := rfl
27   assoc _ _ _ := rfl
28
29 def Emb : Dom ==> Type where
30   obj X := X
31   map f := f
32
33 structure DomIns where
34   toDom : Dom
35
36 instance : CoeSort DomIns Type where coe X := X.toDom.Carrier
37
38 instance : Category DomIns where
39   Hom X Y := Function.Embedding X Y
40   id X := Function.Embedding.refl X
41   comp f g := f.trans g
42   id_comp f := by ext; rfl
43   comp_id f := by ext; rfl
44   assoc f g h := by ext; rfl
45
46 instance {X Y : DomIns} : CoeFun (X --> Y) (fun _ => X -> Y) where
47   coe f := f.toFun
48
49 @[ext]
50 theorem DomIns.hom_ext {X Y : DomIns} (f g : X --> Y)
51   (h : forall x, f x = g x) : f = g := by
52   exact Function.Embedding.ext h
53
54 instance {X Y : DomIns} (f : X --> Y) : Mono f where
55   right_cancellation g h w := by
56     apply DomIns.hom_ext
57     intro x
58     apply f.injective
59     exact congrFun (congrArg Function.Embedding.toFun w) x
60
61 def DomIns.forget : DomIns ==> Dom where
62   obj X := X.toDom
63   map f := f.toFun
64
65 abbrev CoDomIns := Opposite DomIns
66
```

```

67 /-- A chain carries its skeletal well-ordered object type and a proof that
68 its order type lies below  $\omega_0 \sim \omega_0$ . Retaining the carrier explicitly makes
69 ordinal sums of chains definitionally usable. -/
70 structure Chain where
71   Obj : Type
72   linearOrder : LinearOrder Obj
73   wellFoundedLT : WellFoundedLT Obj
74   orderType_lt : Ordinal.type (fun x y : Obj => x < y) <
75     Ordinal.omega0 ~ Ordinal.omega0
76
77 attribute [instance] Chain.linearOrder Chain.wellFoundedLT
78
79 def Chain.sum (X Y : Chain) : Chain where
80   Obj := Lex (Sum X.Obj Y.Obj)
81   linearOrder := inferInstance
82   wellFoundedLT := <|by
83     change WellFounded (Sum.Lex (fun x y : X.Obj => x < y)
84       (fun x y : Y.Obj => x < y))
85     exact Sum.lex_wf wellFounded_lt wellFounded_lt|>
86   orderType_lt := by
87     change Ordinal.type (Sum.Lex (fun x y : X.Obj => x < y)
88       (fun x y : Y.Obj => x < y)) < Ordinal.omega0 ~ Ordinal.omega0
89   rw [Ordinal.type_sum_lex]
90   exact Ordinal.principal_add_omega0_opow Ordinal.omega0
91     X.orderType_lt Y.orderType_lt
92
93 /-- The common page-indexing chain. -/
94 def Spine : Chain where
95   Obj := Nat
96   linearOrder := inferInstance
97   wellFoundedLT := inferInstance
98   orderType_lt := by
99     have hpow : Ordinal.omega0 ~ (1 : Ordinal) <
100       Ordinal.omega0 ~ Ordinal.omega0 :=
101       (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
102     Ordinal.one_lt_omega0
103     change typeLT Nat < Ordinal.omega0 ~ Ordinal.omega0
104     rw [Ordinal.type_nat_lt]
105     simpa only [Ordinal.opow_one] using hpow
106
107 abbrev Con := Chain
108
109 /-- A functor between ordinal chains is equivalently a monotone object map. -/
110 structure ConHom (X Y : Con) where
111   toFun : X.Obj -> Y.Obj
112   monotone : Monotone toFun
113   point_surjective : Function.Surjective toFun
114
115 def ConHom.sum {X X' Y Y' : Con} (f : ConHom X Y) (g : ConHom X' Y') :
116   ConHom (Chain.sum X X') (Chain.sum Y Y') where
117   toFun z := toLex (Sum.map f.toFun g.toFun (ofLex z))
118   monotone := by
119     intro a b h
120     change Sum.Lex (fun x y : X.Obj => x <= y) (fun x y : X'.Obj => x <= y)
121       (ofLex a) (ofLex b) at h
122     change Sum.Lex (fun x y : Y.Obj => x <= y) (fun x y : Y'.Obj => x <= y)
123       (Sum.map f.toFun g.toFun (ofLex a))
124       (Sum.map f.toFun g.toFun (ofLex b))
125     generalize ha : ofLex a = sa at h | -
126     generalize hb : ofLex b = sb at h | -
127     rcases sa with x | x <;> rcases sb with y | y
128     .
129     simp only [Sum.map]
130     exact Sum.Lex.inl (f.monotone (Sum.lex_inl_inl.mp h))
131     . simp only [Sum.map]
132     exact Sum.Lex.sep _ _
133     . exact (Sum.lex_inr_inl h).elim
134     .
135     simp only [Sum.map]
136     exact Sum.Lex.inr (g.monotone (Sum.lex_inr_inr.mp h))
137   point_surjective := by
138     intro z
139     rcases hz : ofLex z with y | y

```



```

140   . obtain <|x, rfl|> := f.point_surjective y
141   refine <|toLex (Sum.inl x), ?_|>
142   apply toLex.injective
143   simp using hz.symm
144   . obtain <|x, rfl|> := g.point_surjective y
145   refine <|toLex (Sum.inr x), ?_|>
146   apply toLex.injective
147   simp using hz.symm
148
149 instance {X Y : Con} : CoeFun (ConHom X Y) (fun _ => X.Obj -> Y.Obj) where
150   coe f := f.toFun
151
152 @[ext]
153 theorem ConHom.ext {X Y : Con} (f g : ConHom X Y)
154   (h : forall x, f x = g x) : f = g := by
155   cases f
156   cases g
157   simp_all only [mk.injEq]
158   exact funext h
159
160 instance : Category.{0} Con where
161   Hom := ConHom
162   id X := <|id, monotone_id, Function.surjective_id|>
163   comp f g := <|g o f, g.monotone.comp f.monotone,
164     g.point_surjective.comp f.point_surjective|>
165   id_comp f := by ext; rfl
166   comp_id f := by ext; rfl
167   assoc f g h := by ext; rfl
168
169 instance {X Y : Con} : CoeFun (X --> Y) (fun _ => X.Obj -> Y.Obj) where
170   coe f := f.toFun
171
172 def Con.asFunctor {X Y : Con} (f : X --> Y) : X.Obj ==> Y.Obj where
173   obj := f
174   map g := homOfLE (f.monotone (leOfHom g))
175
176 abbrev CoCon := Opposite Con
177
178 def Tra : Con ==> Type where
179   obj X := X.Obj
180   map f := f.toFun
181
182 /-- A folio is stored by the least finite presentation of its eventually
183 constant spine functor. `length` is its cardinality: page `length - 1` is
184 the final genuine page and every later spine page repeats it. -/
185 structure Folio where
186   length : Nat
187   positive : 0 < length
188   core : Functor.{0, 0} (Fin length) CoCon
189   originEquiv : (core.obj <|0, positive|>).unop.Obj Equiv Unit
190
191 def Folio.paddedIndex (W : Folio) (n : Nat) : Fin W.length :=
192   <|min n (W.length - 1), lt_of_le_of_lt (min_le_right _ _)
193     (Nat.sub_lt W.positive (by omega))|>
194
195 theorem Folio.paddedIndex_mono (W : Folio) :
196   Monotone W.paddedIndex := by
197   intro a b hab
198   exact min_le_min hab le_rfl
199
200 @[simp]
201 theorem Folio.paddedIndex_fin (W : Folio) (m : Fin W.length) :
202   W.paddedIndex m.1 = m := by
203   apply Fin.ext
204   simp only [Folio.paddedIndex]
205   exact Nat.min_eq_left (by omega)
206
207 def Folio.spineBase (W : Folio) : Nat ==> Fin W.length where
208   obj n := W.paddedIndex n
209   map f := homOfLE (W.paddedIndex_mono (leOfHom f))
210   map_id _ := Subsingleton.elim _ _
211   map_comp _ _ := Subsingleton.elim _ _
212

```

```

213 /-- The actual functor on the spine denoted by the finite presentation. -/
214 def Folio.F (W : Folio) : Nat ==> CoCon := W.spineBase then W.core
215
216 /-- The page functor on the opposite spine. -/
217 def Folio.spineH (W : Folio) : Natop ==> Type := W.F.leftOp then Tra
218
219 def Folio.H (W : Folio) : (Fin W.length)op ==> Type := W.core.leftOp then Tra
220
221 def Folio.E (W : Folio) : Type 0 := W.H.Elements
222
223 instance (W : Folio) : Category.{0} W.E := categoryOfElements W.H
224
225 instance (W : Folio) (m : (Fin W.length)op) : LinearOrder (W.H.obj m) :=
226   (W.core.obj m.unop).unop.linearOrder
227
228 instance (W : Folio) (x y : W.E) : Subsingleton (x --> y) where
229   allEq f g := CategoryOfElements.ext W.H f g (Subsingleton.elim _ _)
230
231 def Folio.originIndex (W : Folio) : Fin W.length := <|0, W.positive|>
232 def Folio.originBase (W : Folio) : (Fin W.length)op := op W.originIndex
233 def Folio.originValue (W : Folio) : W.H.obj W.originBase :=
234   W.originEquiv.symm ()
235 def Folio.originElement (W : Folio) : W.E := <|W.originBase, W.originValue|>
236
237 def Folio.lastIndex (W : Folio) : Fin W.length :=
238   <|W.length - 1, Nat.sub_lt W.positive (by omega)|>
239 def Folio.lastBase (W : Folio) : (Fin W.length)op := op W.lastIndex
240
241 theorem Folio.origin_unique (W : Folio) (x : W.H.obj W.originBase) :
242   x = W.originValue := by
243     apply W.originEquiv.injective
244     exact Subsingleton.elim _ _
245
246 def Folio.baseToOrigin (W : Folio) (m : (Fin W.length)op) :
247   m --> W.originBase := by
248     letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
249     exact (homOfLE (show W.originIndex <= m.unop by
250       change (0 : Fin W.length) <= m.unop
251       exact bot_le)).op
252
253 def Folio.toOrigin (W : Folio) (x : W.E) : x --> W.originElement :=
254   CategoryOfElements.homMk x W.originElement (W.baseToOrigin x.1)
255   (W.origin_unique _)
256
257 /-- The tall presentation is kept internal. A folio's listed cardinality is
258 only a finite presentation of its genuinely `Nat`-indexed spine. -/
259
260 def Folio.TallE (W : Folio) : Type := W.spineH.Elements
261
262 instance (W : Folio) : Category W.TallE := categoryOfElements W.spineH
263
264 instance (W : Folio) (x y : W.TallE) : Subsingleton (x --> y) where
265   allEq f g := CategoryOfElements.ext W.spineH f g (Subsingleton.elim _ _)
266
267 /-- Collapse a tall occurrence to the coherent finite representative used for
268 storage. This is an implementation map, not the page space of the atlas. -/
269 def Folio.collapseElements (W : Folio) : W.TallE ==> W.E where
270   obj x := <|op (W.paddedIndex x.1.unop), x.2|>
271   map {x y} f := CategoryOfElements.homMk _ _
272     (W.spineBase.map f.val.unop).op f.property
273   map_id _ := by
274     apply CategoryOfElements.ext
275     apply Subsingleton.elim
276   map_comp _ _ := by
277     apply CategoryOfElements.ext
278     apply Subsingleton.elim
279
280 /-- Include a stored cell at its actual page into the full spine. -/
281 def Folio.includeElements (W : Folio) : W.E ==> W.TallE where
282   obj x := <|op x.1.unop.1,
283     W.H.map (eqToHom
284       (congrArg op (W.paddedIndex_fin x.1.unop).symm)) x.2|>
285   map {x y} f := by

```

```

286   rcases x with <|<|m|>, k|>
287   rcases y with <|<|n|>, l|>
288   have hnm : n.1 <= m.1 := leOfHom f.val.unop
289   refine CategoryOfElements.homMk _ _ (homOfLE hnm).op ?_
290   simp only [Folio.spineH, Folio.F, Folio.spineBase]
291   change W.H.map _ (W.H.map _ k) = W.H.map _ l
292   rw [<- FunctorToTypes.map_comp_apply,
293       show _ >> _ = f.val >> _ from Subsingleton.elim _ _]
294   rw [FunctorToTypes.map_comp_apply, f.property]
295   map_id _ := by
296     apply CategoryOfElements.ext
297     apply Subsingleton.elim
298   map_comp _ _ := by
299     apply CategoryOfElements.ext
300     apply Subsingleton.elim
301
302 @[simp]
303 theorem Folio.collapse_include (W : Folio) :
304   W.includeElements then W.collapseElements = Unit W.E := by
305   exact CategoryTheory.Functor.ext
306   (F := W.includeElements then W.collapseElements) (G := Functor.id W.E)
307   (fun x => by
308     rcases x with <|<|m|>, k|>
309     refine Functor.Elements.ext _ _ (congrArg op (W.paddedIndex_fin m)) ?_
310     simp [Folio.includeElements, Folio.collapseElements])
311
312 @[simp]
313 theorem Folio.collapse_include_obj (W : Folio) (x : W.E) :
314   W.collapseElements.obj (W.includeElements.obj x) = x := by
315   exact Functor.congr_obj W.collapse_include x
316
317 /-- The finite presentation is a retract of the full spine, but not conversely:
318 collapsing the first repeated page and including it again changes its page
319 index. This records the precise obstruction to treating the two atlas bases
320 as interchangeable through the evident inclusion/collapse comparison. -/
321 theorem Folio.include_collapse_ne_id (W : Folio) :
322   W.collapseElements then W.includeElements != Functor.id W.TallE := by
323   intro h
324   letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
325   let q : (W.core.obj W.lastIndex).unop -->
326     (W.core.obj W.originIndex).unop :=
327     (W.core.map (homOfLE (Fin.zero_le W.lastIndex))).unop
328   let k : (W.core.obj W.lastIndex).unop.Obj :=
329     Classical.choose (q.point_surjective W.originValue)
330   let x : W.TallE := by
331     refine <|op W.length, ?_|>
332     change (W.core.obj (W.paddedIndex W.length)).unop.Obj
333     have hp : W.paddedIndex W.length = W.lastIndex := by
334       apply Fin.ext
335       simp [Folio.paddedIndex, Folio.lastIndex]
336     exact hp.symm transport k
337   have hx := congrArg
338     (fun F : CategoryTheory.Functor W.TallE W.TallE => (F.obj x).1.unop) h
339   have hbad : W.length - 1 = W.length := by
340     simpa [x, Folio.collapseElements, Folio.includeElements,
341           Folio.paddedIndex] using hx
342   exact (ne_of_lt (Nat.sub_lt W.positive (by omega))) hbad
343
344 structure Pag where
345   folio : Folio
346
347 def Pag.H (W : Pag) : (Fin W.folio.length).op ==> Type 0 := W.folio.H
348 def Pag.E (W : Pag) : Type 0 := W.folio.E
349
350 instance (W : Pag) : Category.{0} W.E := categoryOfElements W.H
351
352 instance : Category.{0} Pag where
353   Hom X Y := X.E ==> Y.E
354   id X := Unit X.E
355   comp F G := F then G
356   id_comp _ := rfl
357   comp_id _ := rfl
358   assoc _ _ _ := rfl

```

```

359
360 /-- Internally, an atlas is stored by its least finite presentation. The
361 `Folio.F` construction repeats its final page along the rest of the spine, so
362 the stabilization and morphism-tail clauses in the extracted definition are
363 enforced by construction. -/
364
365 def Pag.cell (P : Pag) (m : (Fin P.folio.length)op) (k : P.H.obj m) : P.E := <|m, k|>
366
367 def Pag.cellToOrigin (P : Pag) (m : (Fin P.folio.length)op)
368   (k : P.H.obj m) : P.cell m k --> P.folio.originElement :=
369   P.folio.toOrigin (P.cell m k)
370
371 def IsPagewiseDisjoint (P : Pag) (G : P.E ==> DomIns) : Prop :=
372   forall (m : (Fin P.folio.length)op) (i j : P.H.obj m), i != j ->
373     forall (x : G.obj (P.cell m i)) (y : G.obj (P.cell m j)),
374       G.map (P.cellToOrigin m i) x != G.map (P.cellToOrigin m j) y
375
376 structure Atl where
377   P : Pag
378   G : P.E ==> DomIns
379   disjoint : IsPagewiseDisjoint P G
380
381 def Atl.H (X : Atl) : (Fin X.P.folio.length)op ==> Type 0 := X.P.H
382 def Atl.E (X : Atl) : Type 0 := X.P.E
383
384 /-- The `n`th page of an atlas on its genuine infinite spine. -/
385 def Atl.page (X : Atl) (n : Nat) : Type := X.P.folio.spineH.obj (op n)
386
387 /-- A cell of the `n`th page, retaining its position on the infinite spine. -/
388 def Atl.pageCell (X : Atl) (n : Nat) (k : X.page n) : X.P.folio.TallE :=
389   <|op n, k|>
390
391 instance (X : Atl) : Category.{0} X.E := categoryOfElements X.H
392
393 structure AtlHom (X Y : Atl) where
394   P : X.E ==> Y.E
395   A : X.G --> P then Y.G
396
397 def AtlHom.identity (X : Atl) : AtlHom X X where
398   P := Unit X.E
399   A := Id X.G
400
401 def AtlHom.comp {X Y Z : Atl} (f : AtlHom X Y) (g : AtlHom Y Z) : AtlHom X Z where
402   P := f.P then g.P
403   A := f.A >> whiskerLeft f.P g.A
404
405 @[ext]
406 theorem AtlHom.ext {X Y : Atl} (f g : AtlHom X Y)
407   (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
408   cases f
409   cases g
410   simp_all
411
412 /-- Heterogeneous extensionality for natural transformations whose target
413 functors have been identified. This keeps dependent transports localized. -/
414 theorem NatTrans.hext_right {C D : Type*} [Category C] [Category D]
415   {F G H : C ==> D} (alpha : F --> G) (beta : F --> H) (h : G = H)
416   (happ : forall X, HEq (alpha.app X) (beta.app X)) : HEq alpha beta := by
417   cases h
418   apply heq_of_eq
419   ext X
420   exact eq_of_heq (happ X)
421
422 /-- Transporting the codomain of an embedding along an equality of objects is
423 heterogeneously equal to the original embedding. This is the small coherence
424 fact needed whenever stability identifies the image of an origin with the
425 target origin. -/
426 theorem embedding_comp_map_eqToHom_heq {C : Type u} [Category C]
427   (G : CategoryTheory.Functor C DomIns) {a b : C} (h : a = b) {X : DomIns}
428   (e : X --> G.obj a) : HEq (e >> G.map (eqToHom h)) e := by
429   cases h
430   simp
431

```

```

432 instance : Category.{0} Atl where
433   Hom := AtlHom
434   id := AtlHom.identity
435   comp := AtlHom.comp
436   id_comp f := by
437     apply AtlHom.ext
438     . rfl
439     . apply heq_of_eq
440     ext x
441     simp [AtlHom.comp, AtlHom.identity]
442   comp_id f := by
443     apply AtlHom.ext
444     . rfl
445     . apply heq_of_eq
446     ext x
447     simp [AtlHom.comp, AtlHom.identity]
448   assoc f g h := by
449     apply AtlHom.ext
450     . rfl
451     . apply heq_of_eq
452     ext x
453     simp [AtlHom.comp]
454
455 def Folio.commonBase (W : Folio) (m n : (Fin W.length)op) :
456   (Fin W.length)op := op (min m.unop n.unop)
457
458 def Folio.toCommonLeft (W : Folio) (m n : (Fin W.length)op) :
459   m --> W.commonBase m n := (homOfLE (min_le_left _ _)).op
460
461 def Folio.toCommonRight (W : Folio) (m n : (Fin W.length)op) :
462   n --> W.commonBase m n := (homOfLE (min_le_right _ _)).op
463
464 def storedElementLT (X : Atl) (x y : X.E) : Prop :=
465   let m := X.P.folio.commonBase x.1 y.1
466   let l : LinearOrder (X.H.obj m) :=
467     (X.P.folio.core.obj m.unop).unop.linearOrder
468   X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
469   X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2
470
471 def storedExtent (A : Atl) : DomIns := A.G.obj A.P.folio.originElement
472
473 def StoredTerritoryIndex (A : Atl) : Type := A.H.obj A.P.folio.lastBase
474
475 def storedTerritory (A : Atl) (k : StoredTerritoryIndex A) : DomIns :=
476   A.G.obj (A.P.cell A.P.folio.lastBase k)
477
478 def storedOriginImage (X : Atl) (x : X.E) (t : X.G.obj x) : storedExtent X :=
479   X.G.map (X.P.folio.toOrigin x) t
480
481 def storedCovered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
482   exists (k : X.H.obj X.P.folio.lastBase)
483     (l : X.G.obj (X.P.cell X.P.folio.lastBase k)),
484     storedOriginImage X x t =
485     storedOriginImage X (X.P.cell X.P.folio.lastBase k) l
486
487 #!/ TallAtlas` is the internal, genuinely infinite-spine presentation. An
488 `Atl` supplies the coherent page data by pulling its finite storage back along
489 `collapseElements`. This distinction deliberately stays out of the extracted
490 mathematical definitions. -/
491
492 structure TallAtlas where
493   H : Natop ==> Type
494   originValue : H.obj (op 0)
495   originUnique : forall x : H.obj (op 0), x = originValue
496   G : H.Elements ==> DomIns
497   cellLT : H.Elements -> H.Elements -> Prop
498   coveredExtent : Set (G.obj <|op 0, originValue|>)
499
500 def TallAtlas.E (X : TallAtlas) : Type := X.H.Elements
501
502 instance (X : TallAtlas) : Category X.E := categoryOfElements X.H
503
504 instance (X : TallAtlas) (x y : X.E) : Subsingleton (x --> y) where

```

```

505 allEq f g := CategoryOfElements.ext X.H f g (Subsingleton.elim _ _)
506
507 def Atl.tallOriginValue (X : Atl) : X.P.folio.spineH.obj (op 0) := by
508   simp [Folio.spineH, Folio.F, Folio.spineBase, Folio.paddedIndex,
509         Folio.originIndex] using X.P.folio.originValue
510
511 def Atl.tall (X : Atl) : TallAtlas where
512   H := X.P.folio.spineH
513   originValue := X.tallOriginValue
514   originUnique x := by
515     let e : X.P.folio.spineH.obj (op 0) Equiv Unit := by
516       simp [Folio.spineH, Folio.F, Folio.spineBase, Folio.paddedIndex,
517             Folio.originIndex] using X.P.folio.originEquiv
518     apply e.injective
519     exact Subsingleton.elim _ _
520   G := X.P.folio.collapseElements then X.G
521   cellLT x y := storedElementLT X
522     (X.P.folio.collapseElements.obj x)
523     (X.P.folio.collapseElements.obj y)
524   coveredExtent t := storedCovered X
525     (X.P.folio.collapseElements.obj
526      <|op 0, X.tallOriginValue|>) t
527
528 def TallAtlas.originElement (X : TallAtlas) : X.E :=
529   <|op 0, X.originValue|>
530
531 def TallAtlas.toOrigin (X : TallAtlas) (x : X.E) : x --> X.originElement :=
532   CategoryOfElements.homMk x X.originElement
533   (homOfLE (Nat.zero_le x.1.unop)).op (X.originUnique _)
534
535 def TallAtlas.covered (X : TallAtlas) (x : X.E) (t : X.G.obj x) : Prop :=
536   X.coveredExtent (X.G.map (X.toOrigin x) t)
537
538 def TallAtlas.extent (X : TallAtlas) : DomIns := X.G.obj X.originElement
539
540 structure TallAtlasHom (X Y : TallAtlas) where
541   P : X.E ==> Y.E
542   A : X.G --> P then Y.G
543
544 def TallAtlasHom.identity (X : TallAtlas) : TallAtlasHom X X where
545   P := Unit X.E
546   A := Id X.G
547
548 def TallAtlasHom.comp {X Y Z : TallAtlas}
549   (f : TallAtlasHom X Y) (g : TallAtlasHom Y Z) : TallAtlasHom X Z where
550   P := f.P then g.P
551   A := f.A >> whiskerLeft f.P g.A
552
553 @[ext]
554 theorem TallAtlasHom.ext {X Y : TallAtlas} (f g : TallAtlasHom X Y)
555   (hP : f.P = g.P) (hA : HEq f.A g.A) : f = g := by
556   cases f
557   cases g
558   simp_all
559
560 instance : Category TallAtlas where
561   Hom := TallAtlasHom
562   id := TallAtlasHom.identity
563   comp := TallAtlasHom.comp
564   id_comp f := by
565     apply TallAtlasHom.ext
566     . rfl
567     . apply heq_of_eq
568     ext x
569     simp [TallAtlasHom.comp, TallAtlasHom.identity]
570   comp_id f := by
571     apply TallAtlasHom.ext
572     . rfl
573     . apply heq_of_eq
574     ext x
575     simp [TallAtlasHom.comp, TallAtlasHom.identity]
576   assoc f g h := by
577     apply TallAtlasHom.ext

```

```

578 . rfl
579 . apply heq_of_eq
580 . ext x
581 . simp [TallAtlasHom.comp]
582
583 /-- An atlas arrow's action on the infinite presentation is derived from its
584 finite pagination and data transformation. The full spine is first collapsed
585 to the coherent stored representative, transformed by `P` and `A`, and then
586 included at the resulting stored page. Thus no independent spine action is
587 part of an atlas morphism. -/
588
589 def AtlHom.tallP {X Y : Atl} (f : X --> Y) :
590   X.tall.E ==> Y.tall.E :=
591   X.P.folio.collapseElements then f.P then Y.P.folio.includeElements
592
593 def AtlHom.tallA {X Y : Atl} (f : X --> Y) :
594   X.tall.G --> f.tallP then Y.tall.G where
595   app x := f.A.app (X.P.folio.collapseElements.obj x) >>
596     Y.G.map (eqToHom (Y.P.folio.collapse_include_obj
597       (f.P.obj (X.P.folio.collapseElements.obj x))).symm)
598   naturality := by
599     intro x y q
600     simp only [AtlHom.tallP, Atl.tall, Functor.comp_obj, Functor.comp_map]
601     rw [← Category.assoc, f.A.naturality, Category.assoc]
602     simp only [Functor.comp_map]
603     have he :
604       Y.G.map (f.P.map (X.P.folio.collapseElements.map q)) >>
605         Y.G.map (eqToHom (Y.P.folio.collapse_include_obj _).symm) =
606       Y.G.map (eqToHom (Y.P.folio.collapse_include_obj _).symm) >>
607         Y.G.map (Y.P.folio.collapseElements.map
608           (Y.P.folio.includeElements.map
609             (f.P.map (X.P.folio.collapseElements.map q)))) := by
610       rw [← Y.G.map_comp, ← Y.G.map_comp]
611       congr 1
612     rw [he]
613     simp only [Category.assoc]
614
615 def AtlHom.tall {X Y : Atl} (f : X --> Y) : X.tall --> Y.tall where
616   P := f.tallP
617   A := f.tallA
618
619 /-- The coherent identity of an infinite presentation. It identifies every
620 repeated occurrence with the stored representative. -/
621 def Atl.coherence (X : Atl) : X.tall --> X.tall :=
622   (AtlHom.identity X).tall
623
624 /-- Deriving the spine action commutes strictly with composition. -/
625 theorem AtlHom.tall_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
626   (f >> g).tall = f.tall >> g.tall := by
627   have hP : (f >> g).tall.P = (f.tall >> g.tall).P := by
628     exact CategoryTheory.Functor.ext
629     (F := (f >> g).tall.P) (G := (f.tall >> g.tall).P)
630     (fun x => by
631       change Z.P.folio.includeElements.obj
632         (g.P.obj (f.P.obj (X.P.folio.collapseElements.obj x))) =
633       Z.P.folio.includeElements.obj
634         (g.P.obj (Y.P.folio.collapseElements.obj
635           (Y.P.folio.includeElements.obj
636             (f.P.obj (X.P.folio.collapseElements.obj x)))))
637       rw [Y.P.folio.collapse_include_obj])
638   apply TallAtlasHom.ext _ _ hP
639   apply NatTrans.hex_right _ _
640   (congrArg (fun P => P then Z.tall.G) hP)
641   intro x
642   let x_0 := X.P.folio.collapseElements.obj x
643   let y_0 := f.P.obj x_0
644   let eY : y_0 = Y.P.folio.collapseElements.obj
645     (Y.P.folio.includeElements.obj y_0) :=
646     (Y.P.folio.collapse_include_obj y_0).symm
647   let z_0 := g.P.obj y_0
648   let eZ : z_0 = Z.P.folio.collapseElements.obj
649     (Z.P.folio.includeElements.obj z_0) :=
650     (Z.P.folio.collapse_include_obj z_0).symm

```

```

651 let y_1 := Y.P.folio.collapseElements.obj
652   (Y.P.folio.includeElements.obj y_0)
653 let z_1 := g.P.obj y_1
654 let eZ_1 : z_1 = Z.P.folio.collapseElements.obj
655   (Z.P.folio.includeElements.obj z_1) :=
656   (Z.P.folio.collapse_include_obj z_1).symm
657 let eR : z_0 = Z.P.folio.collapseElements.obj
658   (Z.P.folio.includeElements.obj z_1) :=
659   (congrArg g.P.obj eY).trans eZ_1
660 change HEq
661   ((AtlHom.comp f g).tallA.app x)
662   ((TallAtlasHom.comp f.tall g.tall).A.app x)
663 dsimp [AtlHom.tall, AtlHom.tallA, AtlHom.tallP, AtlHom.comp,
664   TallAtlasHom.comp]
665 rw [Category.assoc (f.A.app x_0) (Y.G.map (eqToHom eY))
666   (g.A.app y_1 >> Z.G.map (eqToHom eZ_1))]
667 rw [g.A.naturality_assoc (eqToHom eY)]
668 change HEq
669   ((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (eqToHom eZ))
670   (((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (g.P.map (eqToHom eY))) >>
671     Z.G.map (eqToHom eZ_1))
672 have hrArrow : g.P.map (eqToHom eY) >> eqToHom eZ_1 = eqToHom eR :=
673   CategoryOfElements.ext Z.H _ _ (Subsingleton.elim _ _)
674 have hright : HEq
675   (((f.A.app x_0 >> g.A.app y_0) >> Z.G.map (g.P.map (eqToHom eY))) >>
676     Z.G.map (eqToHom eZ_1))
677   (f.A.app x_0 >> g.A.app y_0) := by
678   rw [Category.assoc, <- Z.G.map_comp, hrArrow]
679   exact embedding_comp_map_eqToHom_heq Z.G eR _
680   exact (embedding_comp_map_eqToHom_heq Z.G eZ
681     (f.A.app x_0 >> g.A.app y_0)).trans hright.symm
682
683 theorem Atl.coherence_idempotent (X : Atl) :
684   X.coherence >> X.coherence = X.coherence := by
685   have h := AtlHom.tall_comp (Id X) (Id X)
686   simpa [Atl.coherence] using h.symm
687
688 theorem Atl.coherence_ne_identity (X : Atl) :
689   X.coherence != Id X.tall := by
690   intro h
691   apply X.P.folio.include_collapse_ne_id
692   simpa [Atl.coherence, AtlHom.tall, AtlHom.tallP, AtlHom.identity,
693     TallAtlasHom.identity] using congrArg TallAtlasHom.P h
694
695 theorem AtlHom.coherence_left {X Y : Atl} (f : X --> Y) :
696   X.coherence >> f.tall = f.tall := by
697   have h := AtlHom.tall_comp (Id X) f
698   simpa [Atl.coherence] using h.symm
699
700 theorem AtlHom.coherence_right {X Y : Atl} (f : X --> Y) :
701   f.tall >> Y.coherence = f.tall := by
702   have h := AtlHom.tall_comp f (Id Y)
703   simpa [Atl.coherence] using h.symm
704
705 def cardinality (A : Atl) : Nat := A.P.folio.length
706
707 def extent (A : Atl) : DomIns := storedExtent A
708
709 def TerritoryIndex (A : Atl) : Type := StoredTerritoryIndex A
710
711 def territory (A : Atl) (k : TerritoryIndex A) : DomIns :=
712   storedTerritory A k
713
714 def region (A : Atl) (n : TerritoryIndex A) : DomIns := territory A n
715
716 def IsTransposal : MorphismProperty Atl :=
717   fun _ _ F => Function.Injective F.P.obj
718
719 instance : IsTransposal.IsMultiplicative where
720   id_mem _ := Function.injective_id
721   comp_mem _ _ hf hg := hg.comp hf
722
723 abbrev AtlTrap := WideSubcategory IsTransposal

```



```

724
725 def AtlTrapInc : AtlTrap ==> Atl := wideSubcategoryInclusion IsTransposal
726
727 def elementLT (X : Atl) (x y : X.E) : Prop :=
728   let m := X.P.folio.commonBase x.1 y.1
729   let l : LinearOrder (X.H.obj m) :=
730     (X.P.folio.core.obj m.unop).unop.linearOrder
731   X.H.map (X.P.folio.toCommonLeft x.1 y.1) x.2 <
732   X.H.map (X.P.folio.toCommonRight x.1 y.1) y.2
733
734 /-- The image of a datum in the extent. -/
735 def originImage (X : Atl) (x : X.E) (t : X.G.obj x) : extent X :=
736   storedOriginImage X x t
737
738 /-- A datum is covered when its image in the extent comes from a final region. -/
739 def Covered (X : Atl) (x : X.E) (t : X.G.obj x) : Prop :=
740   storedCovered X x t
741
742 theorem covered_region (X : Atl) (k : TerritoryIndex X) (l : territory X k) :
743   Covered X (X.P.cell X.P.folio.lastBase k) l :=
744   <|k, l, rfl|>
745
746 /-- The order and coverage conditions from Definition 7.2. -/
747 def IsTraversal : MorphismProperty Atl := fun X Y F =>
748   Function.Injective F.P.obj /\
749   (forall x y, elementLT _ x y -> elementLT _ (F.P.obj x) (F.P.obj y)) /\
750   (forall x (t : X.G.obj x), Covered X x t ->
751     Covered Y (F.P.obj x) (F.A.app x t))
752
753 instance : IsTraversal.IsMultiplicative where
754   id_mem X := by
755     refine <|Function.injective_id, fun _ _ h => h, ?_|>
756     intro x t h
757     simpa [AtlHom.identity] using h
758   comp_mem f g hf hg := by
759     refine <|hg.1.comp hf.1, fun x y h => hg.2.1 _ _ (hf.2.1 _ _ h), ?_|>
760     intro x t h
761     simpa [AtlHom.comp] using hg.2.2 (f.P.obj x) (f.A.app x t) (hf.2.2 x t h)
762
763 abbrev AtlTrav := WideSubcategory IsTraversal
764
765 def AtlTravInc : AtlTrav ==> Atl := wideSubcategoryInclusion IsTraversal
766
767 def AtlTravToAtlTrap : AtlTrav ==> AtlTrap where
768   obj X := WideSubcategory.mk X.obj
769   map f := <|f.1, f.2.1|>
770
771 def IsStableTraversal : MorphismProperty AtlTrav := fun X Y F =>
772   F.1.P.obj X.obj.P.folio.originElement = Y.obj.P.folio.originElement
773
774 instance : IsStableTraversal.IsMultiplicative where
775   id_mem _ := rfl
776   comp_mem f g hf hg := by
777     change g.1.P.obj (f.1.P.obj _) = _
778     rw [hf, hg]
779
780 abbrev StaAtlTrav := WideSubcategory IsStableTraversal
781
782 def StaAtlTravToAtlTrav : StaAtlTrav ==> AtlTrav :=
783   wideSubcategoryInclusion IsStableTraversal
784
785 def StaAtlTravInc : StaAtlTrav ==> Atl := StaAtlTravToAtlTrav then AtlTravInc
786
787 /-! `StableAtlasFamily` implements atlas federations of stable atlases.
788 The empty family is the empty atlas operation, and concatenation retains all
789 components without choosing representatives or identifying their data. Most
790 importantly, every component arrow is an arrow of `StaAtlTrav`, so stability is
791 checked by Lean at the boundary of the horizontal construction. -/
792
793 structure StableAtlasFamily where
794   Index : Type
795   countableIndex : Countable Index
796   component : Index -> StaAtlTrav

```

```

797
798 attribute [instance] StableAtlasFamily.countableIndex
799
800 structure StableAtlasFamilyHom (X Y : StableAtlasFamily) where
801   index : X.Index -> Y.Index
802   component : forall i, X.component i --> Y.component (index i)
803
804 /-- Each component of an atlas-federation morphism is stable by construction. -/
805 theorem StableAtlasFamilyHom.component_stable {X Y : StableAtlasFamily}
806   (f : StableAtlasFamilyHom X Y) (i : X.Index) :
807     IsStableTraversal (f.component i).1 :=
808     (f.component i).2
809
810 @[ext]
811 theorem StableAtlasFamilyHom.ext {X Y : StableAtlasFamily}
812   (f g : StableAtlasFamilyHom X Y) (hi : f.index = g.index)
813   (hc : forall i, HEq (f.component i) (g.component i)) : f = g := by
814     cases f
815     cases g
816     cases hi
817     congr
818     funext i
819     exact eq_of_heq (hc i)
820
821 def StableAtlasFamilyHom.identity (X : StableAtlasFamily) :
822   StableAtlasFamilyHom X X where
823   index := id
824   component := fun _ => Id _
825
826 def StableAtlasFamilyHom.comp {X Y Z : StableAtlasFamily}
827   (f : StableAtlasFamilyHom X Y) (g : StableAtlasFamilyHom Y Z) :
828   StableAtlasFamilyHom X Z where
829   index := g.index o f.index
830   component := fun i => f.component i >> g.component (f.index i)
831
832 instance : Category StableAtlasFamily where
833   Hom := StableAtlasFamilyHom
834   id := StableAtlasFamilyHom.identity
835   comp := StableAtlasFamilyHom.comp
836   id_comp f := by
837     apply StableAtlasFamilyHom.ext
838     . rfl
839     . intro i
840     exact heq_of_eq (Category.id_comp _)
841   comp_id f := by
842     apply StableAtlasFamilyHom.ext
843     . rfl
844     . intro i
845     exact heq_of_eq (Category.comp_id _)
846   assoc f g h := by
847     apply StableAtlasFamilyHom.ext
848     . rfl
849     . intro i
850     exact heq_of_eq (Category.assoc _ _ _)
851
852 @[simp]
853 theorem StableAtlasFamilyHom.component_id (X : StableAtlasFamily)
854   (i : X.Index) :
855   (Id X : X --> X).component i = Id (X.component i) := rfl
856
857 @[simp]
858 theorem StableAtlasFamilyHom.component_comp {X Y Z : StableAtlasFamily}
859   (f : X --> Y) (g : Y --> Z) (i : X.Index) :
860   (f >> g).component i = f.component i >> g.component (f.index i) := rfl
861
862 /-- An atlas as a single-component atlas federation. -/
863 def stableAtlasAtom (X : StaAtlTrav) : StableAtlasFamily where
864   Index := Unit
865   countableIndex := inferInstance
866   component := fun _ => X
867
868 /-- The empty atlas federation has no components. -/
869 def stableAtlasUnit : StableAtlasFamily where

```

```

870   Index := Empty
871   countableIndex := inferInstance
872   component := fun i => nomatch i
873
874   /-- Federation merge is disjoint tagged retention of components,
875   not a categorical coproduct of atlases. -/
876   def stableAtlasMerge (X Y : StableAtlasFamily) : StableAtlasFamily where
877     Index := Sum X.Index Y.Index
878     countableIndex := inferInstance
879     component := Sum.elim X.component Y.component
880
881   def stableAtlasMergeHom {X_1 X_2 Y_1 Y_2 : StableAtlasFamily}
882     (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
883     stableAtlasMerge X_1 X_2 --> stableAtlasMerge Y_1 Y_2 where
884     index := Sum.map f.index g.index
885     component
886     | .inl i => f.component i
887     | .inr j => g.component j
888
889   /-- Horizontal sum on atlas federations. -/
890   def StableAtlHorSum : StableAtlasFamily * StableAtlasFamily ==>
891     StableAtlasFamily where
892     obj X := stableAtlasMerge X.1 X.2
893     map f := stableAtlasMergeHom f.1 f.2
894     map_id X := by
895       apply StableAtlasFamilyHom.ext
896       . funext i
897       cases i <=> rfl
898       . intro i
899       cases i <=> exact HEq.rfl
900     map_comp f g := by
901       apply StableAtlasFamilyHom.ext
902       . funext i
903       cases i <=> rfl
904       . intro i
905       cases i <=> exact HEq.rfl
906
907   def stableAtlasAssociatorHom (X Y Z : StableAtlasFamily) :
908     stableAtlasMerge (stableAtlasMerge X Y) Z -->
909     stableAtlasMerge X (stableAtlasMerge Y Z) where
910     index
911     | .inl (.inl i) => .inl i
912     | .inl (.inr j) => .inr (.inl j)
913     | .inr k => .inr (.inr k)
914     component
915     | .inl (.inl i) => Id (X.component i)
916     | .inl (.inr j) => Id (Y.component j)
917     | .inr k => Id (Z.component k)
918
919   def stableAtlasAssociatorInv (X Y Z : StableAtlasFamily) :
920     stableAtlasMerge X (stableAtlasMerge Y Z) -->
921     stableAtlasMerge (stableAtlasMerge X Y) Z where
922     index
923     | .inl i => .inl (.inl i)
924     | .inr (.inl j) => .inl (.inr j)
925     | .inr (.inr k) => .inr k
926     component
927     | .inl i => Id (X.component i)
928     | .inr (.inl j) => Id (Y.component j)
929     | .inr (.inr k) => Id (Z.component k)
930
931   def stableAtlasAssociator (X Y Z : StableAtlasFamily) :
932     stableAtlasMerge (stableAtlasMerge X Y) Z Iso
933     stableAtlasMerge X (stableAtlasMerge Y Z) where
934     hom := stableAtlasAssociatorHom X Y Z
935     inv := stableAtlasAssociatorInv X Y Z
936     hom_inv_id := by
937       apply StableAtlasFamilyHom.ext
938       . funext i
939       rcases i with (i | j) | k <=> rfl
940       . intro i
941       rcases i with (i | j) | k <=>
942         simp [stableAtlasAssociatorHom, stableAtlasAssociatorInv,

```

```

943     stableAtlasMerge]
944 inv_hom_id := by
945   apply StableAtlasFamilyHom.ext
946   . funext i
947     rcases i with i | (j | k) <;> rfl
948   . intro i
949     rcases i with i | (j | k) <;>
950     simp [stableAtlasAssociatorHom, stableAtlasAssociatorInv,
951           stableAtlasMerge]
952
953 def stableAtlasLeftUnitorHom (X : StableAtlasFamily) :
954   stableAtlasMerge stableAtlasUnit X --> X where
955   index
956   | .inr i => i
957   component
958   | .inr i => Id (X.component i)
959
960 def stableAtlasLeftUnitorInv (X : StableAtlasFamily) :
961   X --> stableAtlasMerge stableAtlasUnit X where
962   index i := .inr i
963   component i := Id (X.component i)
964
965 def stableAtlasLeftUnitor (X : StableAtlasFamily) :
966   stableAtlasMerge stableAtlasUnit X Iso X where
967   hom := stableAtlasLeftUnitorHom X
968   inv := stableAtlasLeftUnitorInv X
969   hom_inv_id := by
970     apply StableAtlasFamilyHom.ext
971     . funext i
972       rcases i with i | i
973       . exact i.elim
974       . rfl
975     . intro i
976       rcases i with i | i
977       . exact i.elim
978       . simp [stableAtlasLeftUnitorHom, stableAtlasLeftUnitorInv,
979             stableAtlasMerge]
980   inv_hom_id := by
981     apply StableAtlasFamilyHom.ext
982     . rfl
983     . intro i
984       simp [stableAtlasLeftUnitorHom, stableAtlasLeftUnitorInv,
985             stableAtlasMerge]
986
987 def stableAtlasRightUnitorHom (X : StableAtlasFamily) :
988   stableAtlasMerge X stableAtlasUnit --> X where
989   index
990   | .inl i => i
991   component
992   | .inl i => Id (X.component i)
993
994 def stableAtlasRightUnitorInv (X : StableAtlasFamily) :
995   X --> stableAtlasMerge X stableAtlasUnit where
996   index i := .inl i
997   component i := Id (X.component i)
998
999 def stableAtlasRightUnitor (X : StableAtlasFamily) :
1000   stableAtlasMerge X stableAtlasUnit Iso X where
1001   hom := stableAtlasRightUnitorHom X
1002   inv := stableAtlasRightUnitorInv X
1003   hom_inv_id := by
1004     apply StableAtlasFamilyHom.ext
1005     . funext i
1006       rcases i with i | i
1007       . rfl
1008       . exact i.elim
1009     . intro i
1010       rcases i with i | i
1011       . simp [stableAtlasRightUnitorHom, stableAtlasRightUnitorInv,
1012             stableAtlasMerge]
1013       . exact i.elim
1014   inv_hom_id := by
1015     apply StableAtlasFamilyHom.ext

```

```

1016   . rfl
1017   . intro i
1018   simp [stableAtlasRightUnitorHom, stableAtlasRightUnitorInv,
1019         stableAtlasMerge]
1020
1021 def stableAtlasBraidingHom (X Y : StableAtlasFamily) :
1022   stableAtlasMerge X Y --> stableAtlasMerge Y X where
1023   index
1024   | .inl i => .inr i
1025   | .inr j => .inl j
1026   component
1027   | .inl i => Id (X.component i)
1028   | .inr j => Id (Y.component j)
1029
1030 def stableAtlasBraiding (X Y : StableAtlasFamily) :
1031   stableAtlasMerge X Y Iso stableAtlasMerge Y X where
1032   hom := stableAtlasBraidingHom X Y
1033   inv := stableAtlasBraidingHom Y X
1034   hom_inv_id := by
1035     apply StableAtlasFamilyHom.ext
1036     . funext i
1037     cases i <|> rfl
1038     . intro i
1039     cases i <|> simp [stableAtlasBraidingHom, stableAtlasMerge]
1040   inv_hom_id := by
1041     apply StableAtlasFamilyHom.ext
1042     . funext i
1043     cases i <|> rfl
1044     . intro i
1045     cases i <|> simp [stableAtlasBraidingHom, stableAtlasMerge]
1046
1047 instance stableAtlasMonoidalStruct : MonoidalCategoryStruct StableAtlasFamily where
1048   tensorObj := stableAtlasMerge
1049   tensorHom := stableAtlasMergeHom
1050   whiskerLeft X _ f := stableAtlasMergeHom (Id X) f
1051   whiskerRight f Y := stableAtlasMergeHom f (Id Y)
1052   tensorUnit := stableAtlasUnit
1053   associator := stableAtlasAssociator
1054   leftUnitor := stableAtlasLeftUnitor
1055   rightUnitor := stableAtlasRightUnitor
1056
1057 instance stableAtlasMonoidal : MonoidalCategory StableAtlasFamily :=
1058   MonoidalCategory.ofTensorHom
1059   (id_tensorHom_id := fun X Y => StableAtlasHorSum.map_id (X, Y))
1060   (id_tensorHom := fun X { _ } f => rfl)
1061   (tensorHom_id := fun { _ } f Y => rfl)
1062   (tensorHom_comp_tensorHom := fun {X_1 Y_1 Z_1 X_2 Y_2 Z_2} f_1 f_2 g_1 g_2 => by
1063     apply StableAtlasFamilyHom.ext
1064     . funext i
1065     cases i <|> rfl
1066     . intro i
1067     cases i <|> simp [stableAtlasMonoidalStruct, stableAtlasMergeHom])
1068   (associator_naturality := fun { _ _ _ _ } f_1 f_2 f_3 => by
1069     apply StableAtlasFamilyHom.ext
1070     . funext i
1071     rcases i with (i | j) | k <|> rfl
1072     . intro i
1073     rcases i with (i | j) | k <|>
1074       simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1075             stableAtlasAssociator, stableAtlasAssociatorHom, stableAtlasMerge])
1076   (leftUnitor_naturality := fun { _ } f => by
1077     apply StableAtlasFamilyHom.ext
1078     . funext i
1079     rcases i with i | i
1080     . exact i.elim
1081     . rfl
1082     . intro i
1083     rcases i with i | i
1084     . exact i.elim
1085     . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1086             stableAtlasLeftUnitor, stableAtlasLeftUnitorHom, stableAtlasMerge])
1087   (rightUnitor_naturality := fun { _ } f => by
1088     apply StableAtlasFamilyHom.ext

```

```

1089     . funext i
1090     rcases i with i | i
1091     . rfl
1092     . exact i.elim
1093   . intro i
1094   rcases i with i | i
1095   . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1096     stableAtlasRightUnitor, stableAtlasRightUnitorHom, stableAtlasMerge]
1097   . exact i.elim)
1098 (pentagon := fun W X Y Z => by
1099   apply StableAtlasFamilyHom.ext
1100   . funext i
1101   rcases i with ((i | j) | k) | l <=> rfl
1102   . intro i
1103   rcases i with ((i | j) | k) | l <=>
1104     simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1105       stableAtlasAssociator, stableAtlasAssociatorHom, stableAtlasMerge])
1106 (triangle := fun X Y => by
1107   apply StableAtlasFamilyHom.ext
1108   . funext i
1109   rcases i with (i | e) | j
1110   . rfl
1111   . exact e.elim
1112   . rfl
1113   . intro i
1114   rcases i with (i | e) | j
1115   . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1116     stableAtlasAssociator, stableAtlasAssociatorHom,
1117     stableAtlasRightUnitor, stableAtlasRightUnitorHom, stableAtlasMerge]
1118   . exact e.elim
1119   . simp [stableAtlasMonoidalStruct, stableAtlasMergeHom,
1120     stableAtlasAssociator, stableAtlasAssociatorHom,
1121     stableAtlasLeftUnitor, stableAtlasLeftUnitorHom, stableAtlasMerge])
1122
1123 @[simp]
1124 theorem stableAtlas_tensorObj (X Y : StableAtlasFamily) :
1125   MonoidalCategoryStruct.tensorObj X Y = stableAtlasMerge X Y := rfl
1126
1127 @[simp]
1128 theorem stableAtlas_tensorHom {X_1 Y_1 X_2 Y_2 : StableAtlasFamily}
1129   (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
1130   MonoidalCategoryStruct.tensorHom f g = stableAtlasMergeHom f g := rfl
1131
1132 @[simp]
1133 theorem stableAtlas_whiskerLeft (X : StableAtlasFamily)
1134   {Y_1 Y_2 : StableAtlasFamily} (f : Y_1 --> Y_2) :
1135   MonoidalCategoryStruct.whiskerLeft X f =
1136     stableAtlasMergeHom (Id X) f := rfl
1137
1138 @[simp]
1139 theorem stableAtlas_whiskerRight {X_1 X_2 : StableAtlasFamily}
1140   (f : X_1 --> X_2) (Y : StableAtlasFamily) :
1141   MonoidalCategoryStruct.whiskerRight f Y =
1142     stableAtlasMergeHom f (Id Y) := rfl
1143
1144 @[simp]
1145 theorem stableAtlas_associator (X Y Z : StableAtlasFamily) :
1146   MonoidalCategoryStruct.associator X Y Z =
1147     stableAtlasAssociator X Y Z := rfl
1148
1149 instance stableAtlasBraided : BraidedCategory StableAtlasFamily where
1150   braiding := stableAtlasBraiding
1151   braiding_naturality_right := fun X { _ _ } f => by
1152     apply StableAtlasFamilyHom.ext
1153     . funext i
1154     cases i <=> rfl
1155     . intro i
1156     cases i <=>
1157       simp [stableAtlasMergeHom, stableAtlasBraiding,
1158         stableAtlasBraidingHom, stableAtlasMerge]
1159   braiding_naturality_left := fun { _ _ } f Z => by
1160     apply StableAtlasFamilyHom.ext
1161     . funext i

```

```

1162     cases i <;> rfl
1163   . intro i
1164     cases i <;>
1165       simp [stableAtlasMergeHom, stableAtlasBraiding,
1166             stableAtlasBraidingHom, stableAtlasMerge]
1167   hexagon_forward := fun X Y Z => by
1168     apply StableAtlasFamilyHom.ext
1169   . funext i
1170     rcases i with (i | j) | k <;> rfl
1171   . intro i
1172     rcases i with (i | j) | k <;>
1173       simp [stableAtlasMergeHom, stableAtlasAssociator,
1174             stableAtlasAssociatorHom, stableAtlasBraiding,
1175             stableAtlasBraidingHom, stableAtlasMerge]
1176   hexagon_reverse := fun X Y Z => by
1177     apply StableAtlasFamilyHom.ext
1178   . funext i
1179     rcases i with i | (j | k) <;> rfl
1180   . intro i
1181     rcases i with i | (j | k) <;>
1182       simp [stableAtlasMergeHom, stableAtlasAssociator,
1183             stableAtlasAssociatorInv, stableAtlasBraiding,
1184             stableAtlasBraidingHom, stableAtlasMerge]
1185
1186 instance : SymmetricCategory StableAtlasFamily where
1187   symmetry X Y := by
1188     apply StableAtlasFamilyHom.ext
1189   . funext i
1190     cases i <;> rfl
1191   . intro i
1192     cases i <;>
1193       simp [stableAtlasBraided, stableAtlasBraiding,
1194             stableAtlasBraidingHom, stableAtlasMerge]
1195
1196 def IsAtlasMap : ObjectProperty Atl := fun A =>
1197   forall v : extent A, Covered A A.P.folio.originElement v
1198
1199 abbrev AtlMap := IsAtlasMap.FullSubcategory
1200
1201 def AtlMapInc : AtlMap ==> Atl := ObjectProperty.iota IsAtlasMap
1202
1203 def IsAtlasMapTrav : ObjectProperty AtlTrav := fun A => IsAtlasMap A.obj
1204 abbrev AtlTravMap := IsAtlasMapTrav.FullSubcategory
1205
1206 def AtlTravMapInc : AtlTravMap ==> AtlTrav := ObjectProperty.iota IsAtlasMapTrav
1207
1208 /-- The dominion of covered data in a cell. -/
1209 def coveredDom (X : Atl) (x : X.E) : DomIns where
1210   toDom :=
1211     { Carrier := {t : X.G.obj x // Covered X x t}
1212     rank :=
1213       ({ toFun := Subtype.val
1214         inj' := Subtype.val_injective } :
1215         Function.Embedding {t : X.G.obj x // Covered X x t} (X.G.obj x)).trans
1216         (X.G.obj x).toDom.rank }
1217
1218 def coveredMap {X : Atl} {x y : X.E} (f : x --> y) :
1219   coveredDom X x --> coveredDom X y where
1220   toFun t := <|X.G.map f t.1, by
1221     rcases t.2 with <|k, l, h|>
1222     refine <|k, l, ?_|>
1223     have e : f >> X.P.folio.toOrigin y = X.P.folio.toOrigin x :=
1224       CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
1225     change (X.G.map f >> X.G.map (X.P.folio.toOrigin y)) t.1 = _
1226     rw [<- X.G.map_comp, e]
1227     exact h|>
1228   inj' := fun a b h => Subtype.ext <| (X.G.map f).injective <| congrArg Subtype.val h
1229
1230 def coveredFunctor (X : Atl) : X.E ==> DomIns where
1231   obj := coveredDom X
1232   map := coveredMap
1233   map_id x := by
1234     apply DomIns.hom_ext

```

```

1235     intro t
1236     apply Subtype.ext
1237     change (X.G.map (Id x)) t.1 = t.1
1238     rw [X.G.map_id]
1239     change Function.Embedding.refl _ t.1 = t.1
1240     rfl
1241 map_comp f g := by
1242   apply DomIns.hom_ext
1243   intro t
1244   apply Subtype.ext
1245   change (X.G.map (f >> g)) t.1 =
1246     (X.G.map f >> X.G.map g) t.1
1247   rw [X.G.map_comp]
1248
1249 def chartAtlas (X : Atl) : Atl where
1250   P := X.P
1251   G := coveredFunctor X
1252   disjoint := by
1253     intro m i j hij x y h
1254     apply X.disjoint m i j hij x.1 y.1
1255     exact congrArg Subtype.val h
1256
1257 theorem chart_all_covered (X : Atl) (x : (chartAtlas X).E)
1258   (t : (chartAtlas X).G.obj x) : Covered (chartAtlas X) x t := by
1259   rcases t.2 with <|k, l, h|>
1260   let l' : territory (chartAtlas X) k := <|l, covered_region X k l|>
1261   exact <|k, l', Subtype.ext h|>
1262
1263 theorem chart_isAtlasMap (X : Atl) : IsAtlasMap (chartAtlas X) :=
1264   fun v => chart_all_covered X _ v
1265
1266 def chartCounit (X : Atl) : chartAtlas X --> X where
1267   P := Unit X.E
1268   A :=
1269     { app := fun _ =>
1270       { toFun := Subtype.val
1271         inj' := Subtype.val_injective }
1272       naturality := by intros; ext; rfl }
1273
1274 def chartMap {X Y : AtlTrav} (f : X --> Y) :
1275   chartAtlas X.obj --> chartAtlas Y.obj where
1276   P := f.1.P
1277   A :=
1278     { app := fun x =>
1279       { toFun := fun t => <|f.1.A.app x t.1, f.2.2.2 x t.1 t.2|>
1280         inj' := fun a b h => Subtype.ext <| (f.1.A.app x).injective <|
1281           congrArg Subtype.val h }
1282       naturality := by
1283         intro x y g
1284         apply DomIns.hom_ext
1285         intro t
1286         apply Subtype.ext
1287         exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t.1 }
1288
1289 theorem chartMap_isTraversal {X Y : AtlTrav} (f : X --> Y) :
1290   IsTraversal (chartMap f) := by
1291   refine <|f.2.1, f.2.2.1, ?_|>
1292   intro x t _
1293   exact chart_all_covered Y.obj _ _
1294
1295 def Chr : AtlTrav ==> AtlTravMap where
1296   obj X := <|WideSubcategory.mk (chartAtlas X.obj), chart_isAtlasMap X.obj|>
1297   map f := <|chartMap f, chartMap_isTraversal f|>
1298   map_id _ := by
1299     apply Subtype.ext
1300     change chartMap (Id _) = AtlHom.identity _
1301     apply AtlHom.ext
1302     . rfl
1303     . apply heq_of_eq
1304     ext x t
1305     rfl
1306   map_comp f g := by
1307     apply Subtype.ext

```



```

1308   change chartMap (_ >> _) = AtlHom.comp (chartMap _) (chartMap _)
1309   apply AtlHom.ext
1310   . rfl
1311   . apply heq_of_eq
1312     ext x t
1313     rfl
1314
1315   /-- The formal construction realizes the proof sketch by retaining in each
1316   cell exactly the data covered by final regions; its counit is the canonical
1317   inclusion into the original atlas. -/
1318
1319   theorem originImage_origin (X : Atl) (v : extent X) :
1320     originImage X X.P.folio.originElement v = v := by
1321     have e : X.P.folio.toOrigin X.P.folio.originElement =
1322       Id X.P.folio.originElement :=
1323       CategoryOfElements.ext _ _ (Subsingleton.elim _ _)
1324     rw [originImage, storedOriginImage, e, X.G.map_id]
1325     change Function.Embedding.refl _ v = v
1326     rfl
1327
1328   theorem atlasMap_all_covered {X : Atl} (hX : IsAtlasMap X)
1329     (x : X.E) (t : X.G.obj x) : Covered X x t := by
1330     rcases hX (originImage X x t) with <|k, l, h|>
1331     exact <|k, l, by
1332       change originImage X X.P.folio.originElement (originImage X x t) =
1333       originImage X (X.P.cell X.P.folio.lastBase k) l at h
1334       simpa only [originImage_origin] using h|>
1335
1336   theorem chartCounit_isTraversal (X : Atl) : IsTraversal (chartCounit X) := by
1337     refine <|Function.injective_id, fun _ _ h => h, ?_|>
1338     intro x t _
1339     simpa [chartCounit] using t.2
1340
1341   def chartCounitTrav (X : AtlTrav) :
1342     (WideSubcategory.mk (chartAtlas X.obj) : AtlTrav) --> X :=
1343     <|chartCounit X.obj, chartCounit_isTraversal X.obj|>
1344
1345   def chartLift {A : AtlTravMap} {X : AtlTrav}
1346     (f : AtlTravMapInc.obj A --> X) :
1347     A --> Chr.obj X := by
1348     refine <|?, ?_|>
1349     . exact
1350       { P := f.1.P
1351       A :=
1352         { app := fun x =>
1353           { toFun := fun t => <|f.1.A.app x t,
1354             f.2.2.2 x t (atlasMap_all_covered A.property x t)|>
1355           inj' := by
1356             intro a b h
1357             apply (f.1.A.app x).injective
1358             exact congrArg (fun z => z.1) h }
1359         naturality := by
1360           intro x y g
1361           apply DomIns.hom_ext
1362           intro t
1363           apply Subtype.ext
1364           exact congrFun (congrArg Function.Embedding.toFun (f.1.A.naturality g)) t } }
1365     . refine <|f.2.1, f.2.2.1, ?_|>
1366     intro x t _
1367     exact chart_all_covered X.obj _ _
1368
1369   def chartHomEquiv (A : AtlTravMap) (X : AtlTrav) :
1370     (AtlTravMapInc.obj A --> X) Equiv (A --> Chr.obj X) where
1371     toFun := chartLift
1372     invFun g := g >> chartCounitTrav X
1373     left_inv f := by
1374       apply Subtype.ext
1375       apply AtlHom.ext
1376       . rfl
1377       . apply heq_of_eq
1378         ext x t
1379         rfl
1380     right_inv g := by

```

```

1381   apply Subtype.ext
1382   apply AtlHom.ext
1383   . rfl
1384   . apply heq_of_eq
1385     ext x t
1386     apply Subtype.ext
1387     rfl
1388
1389 theorem chartHomEquiv_naturality_left {A A' : AtlTravMap} (f : A' --> A)
1390   {X : AtlTrav} (g : AtlTravMapInc.obj A --> X) :
1391   chartHomEquiv A' X (AtlTravMapInc.map f >> g) =
1392   f >> chartHomEquiv A X g := by
1393   apply Subtype.ext
1394   apply AtlHom.ext
1395   . rfl
1396   . apply heq_of_eq
1397     ext x t
1398     apply Subtype.ext
1399     rfl
1400
1401 theorem chartHomEquiv_naturality_right {A : AtlTravMap} {X X' : AtlTrav}
1402   (f : AtlTravMapInc.obj A --> X) (g : X --> X') :
1403   chartHomEquiv A X' (f >> g) = chartHomEquiv A X f >> Chr.map g := by
1404   apply Subtype.ext
1405   apply AtlHom.ext
1406   . rfl
1407   . apply heq_of_eq
1408     ext x t
1409     apply Subtype.ext
1410     rfl
1411
1412 def cartographyAdjunction : AtlTravMapInc -| Chr :=
1413   Adjunction.mkOfHomEquiv
1414   { homEquiv := chartHomEquiv
1415     homEquiv_naturality_left_symm := by
1416       intro X' X Y f g
1417       apply (chartHomEquiv X' Y).injective
1418       simp using (chartHomEquiv_naturality_left f ((chartHomEquiv X Y).symm g)).symm
1419     homEquiv_naturality_right := by
1420       intro X Y Y' f g
1421       exact chartHomEquiv_naturality_right f g }
1422
1423 theorem cartographyLemma : Nonempty (AtlTravMapInc -| Chr) :=
1424   <|cartographyAdjunction|>
1425
1426 def extentMap {X Y : Atl} (f : X --> Y) : extent X --> extent Y :=
1427   f.A.app X.P.folio.originElement >>
1428   Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement))
1429
1430 theorem extentMap_originImage {X Y : Atl} (f : X --> Y) (x : X.E)
1431   (a : X.G.obj x) :
1432   extentMap f (originImage X x a) =
1433   originImage Y (f.P.obj x) (f.A.app x a) := by
1434   simp only [extentMap, originImage, storedOriginImage]
1435   have hn := f.A.naturality (X.P.folio.toOrigin x)
1436   have hc : f.P.map (X.P.folio.toOrigin x) >>
1437     Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement) =
1438     Y.P.folio.toOrigin (f.P.obj x) := by
1439     apply CategoryOfElements.ext
1440     apply Subsingleton.elim
1441   rw [← hc, Y.G.map_comp]
1442   exact congrFun (congrArg Function.Embedding.toFun
1443     (congrArg (fun k => k >>
1444       Y.G.map (Y.P.folio.toOrigin (f.P.obj X.P.folio.originElement)))) hn) a
1445
1446 theorem extentMap_id (X : Atl) : extentMap (Id X) = Id (extent X) := by
1447   apply DomIns.hom_ext
1448   intro t
1449   change (X.G.map (X.P.folio.toOrigin X.P.folio.originElement)) t = t
1450   simp [originImage] using originImage_origin X t
1451
1452 theorem extentMap_comp {X Y Z : Atl} (f : X --> Y) (g : Y --> Z) :
1453   extentMap (f >> g) = extentMap f >> extentMap g := by

```

```

1454 apply DomIns.hom_ext
1455 intro t
1456 let oX := X.P.folio.originElement
1457 let y := f.P.obj oX
1458 let oY := Y.P.folio.originElement
1459 let z := g.P.obj y
1460 let z_0 := g.P.obj oY
1461 have hn := g.A.naturality (Y.P.folio.toOrigin y)
1462 have hz : g.P.map (Y.P.folio.toOrigin y) >> Z.P.folio.toOrigin z_0 =
1463   Z.P.folio.toOrigin z :=
1464     CategoryOfElements.ext _ _ _ (Subsingleton.elim _ _)
1465 have hn' : Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY =
1466   g.A.app y >> Z.G.map (g.P.map (Y.P.folio.toOrigin y)) := hn
1467 have hm : g.A.app y >> Z.G.map (Z.P.folio.toOrigin z) =
1468   Y.G.map (Y.P.folio.toOrigin y) >> g.A.app oY >>
1469     Z.G.map (Z.P.folio.toOrigin z_0) := by
1470   rw [← Category.assoc, hn', Category.assoc, ← Z.G.map_comp, hz]
1471 exact congrFun (congrArg Function.Embedding.toFun
1472   (by simp only [Category.assoc] using congrArg (fun q => f.A.app oX >> q) hm)) t
1473
1474 def Ex : Atl ==> DomIns where
1475   obj := extent
1476   map := extentMap
1477   map_id := extentMap_id
1478   map_comp := extentMap_comp
1479
1480 def stableCoalitionMap {X Y : StaAtlTrav} (f : X --> Y) :
1481   coveredDom X.obj.obj X.obj.obj.P.folio.originElement -->
1482   coveredDom Y.obj.obj Y.obj.obj.P.folio.originElement where
1483   toFun t := by
1484     have hc := f.1.2.2.2 X.obj.obj.P.folio.originElement t.1 t.2
1485     exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
1486       <|f.1.1.A.app X.obj.obj.P.folio.originElement t.1, hc|>
1487   inj' := fun a b h => by
1488     apply Subtype.ext
1489     apply (f.1.1.A.app X.obj.obj.P.folio.originElement).injective
1490     apply (Y.obj.obj.G.map (eqToHom f.2)).injective
1491     exact congrArg Subtype.val h
1492
1493 theorem stableCoalitionMap_val {X Y : StaAtlTrav} (f : X --> Y)
1494   (t : coveredDom X.obj.obj X.obj.obj.P.folio.originElement) :
1495   (stableCoalitionMap f t).1 = extentMap f.1.1 t.1 := by
1496   change Y.obj.obj.G.map (eqToHom f.2)
1497     (f.1.1.A.app X.obj.obj.P.folio.originElement t.1) =
1498     Y.obj.obj.G.map (Y.obj.obj.P.folio.toOrigin
1499       (f.1.1.P.obj X.obj.obj.P.folio.originElement))
1500     (f.1.1.A.app X.obj.obj.P.folio.originElement t.1)
1501   have hmor : eqToHom f.2 = Y.obj.obj.P.folio.toOrigin
1502     (f.1.1.P.obj X.obj.obj.P.folio.originElement) := by
1503     apply CategoryOfElements.ext
1504     apply Subsingleton.elim
1505   rw [hmo]
1506
1507 /-- `Coa` is written directly on the covered origins. This is definitionally
1508 the object part of `Ex o Chr`; stability makes its action on arrows reduce to
1509 the component at the origin. -/
1510 def Coa : StaAtlTrav ==> DomIns where
1511   obj X := coveredDom X.obj.obj X.obj.obj.P.folio.originElement
1512   map := stableCoalitionMap
1513   map_id X := by
1514     apply DomIns.hom_ext
1515     intro t
1516     apply Subtype.ext
1517     rw [stableCoalitionMap_val]
1518     exact congrFun (congrArg Function.Embedding.toFun (extentMap_id X.obj.obj)) t.1
1519   map_comp f g := by
1520     apply DomIns.hom_ext
1521     intro t
1522     apply Subtype.ext
1523     rw [stableCoalitionMap_val]
1524     change extentMap (f.1.1 >> g.1.1) t.1 =
1525       (stableCoalitionMap g (stableCoalitionMap f t)).1
1526     rw [stableCoalitionMap_val, stableCoalitionMap_val]

```

```

1527     exact congrFun (congrArg Function.Embedding.toFun
1528       (extentMap_comp f.1.1 g.1.1)) t.1
1529
1530 def coalition (X : Atl) : DomIns := (Coa.obj
1531   (WideSubcategory.mk (WideSubcategory.mk X) : StaAtlTrav))
1532
1533 /-- The initial dominion. -/
1534 def emptyDominion : DomIns where
1535   toDom :=
1536     { Carrier := Empty
1537       rank :=
1538         { toFun := fun x => nomatch x
1539           inj' := fun x => nomatch x } }
1540
1541 def singletonChain : Chain where
1542   Obj := Unit
1543   linearOrder := inferInstance
1544   wellFoundedLT := inferInstance
1545   orderType_lt := by
1546     have hpow : Ordinal.omega0 ^ (1 : Ordinal) <
1547       Ordinal.omega0 ^ Ordinal.omega0 :=
1548       (Ordinal.opow_lt_opow_iff_right Ordinal.one_lt_omega0).2
1549     Ordinal.one_lt_omega0
1550   have homega : Ordinal.omega0 < Ordinal.omega0 ^ Ordinal.omega0 := by
1551     simpa only [Ordinal.opow_one] using hpow
1552   simpa using lt_trans Ordinal.one_lt_omega0 homega
1553
1554 def singletonObjEquiv : singletonChain.Obj Equiv Unit := Equiv.refl _
1555
1556 def onePageFolio : Folio where
1557   length := 1
1558   positive := by omega
1559   core := (Functor.const (Fin 1)).obj (op singletonChain)
1560   originEquiv := singletonObjEquiv
1561
1562 def onePagePag : Pag := <|onePageFolio|>
1563
1564 theorem onePage_cell_unique (x : onePagePag.E) :
1565   x = onePageFolio.originElement := by
1566   let hbase : x.1 = onePageFolio.originBase := by
1567     apply unop_injective
1568     apply Fin.eq_zero
1569   apply Functor.Elements.ext x onePageFolio.originElement hbase
1570   exact onePageFolio.origin_unique _
1571
1572 def dominionAtlas (X : DomIns) : Atl where
1573   P := onePagePag
1574   G := (Functor.const onePagePag.E).obj X
1575   disjoint := by
1576     intro m i j hij
1577     exfalso
1578     apply hij
1579     apply onePageFolio.originEquiv.injective
1580     exact Subsingleton.elim _ _
1581
1582 theorem dominionAtlas_isMap (X : DomIns) : IsAtlasMap (dominionAtlas X) := by
1583   intro v
1584   let k : TerritoryIndex (dominionAtlas X) :=
1585     onePageFolio.originValue
1586   refine <|k, v, ?_|>
1587   change originImage (dominionAtlas X)
1588   (dominionAtlas X).P.folio.originElement v =
1589     originImage (dominionAtlas X)
1590     ((dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k) v
1591   rw [originImage_origin]
1592   have hc : (dominionAtlas X).P.cell (dominionAtlas X).P.folio.lastBase k =
1593     (dominionAtlas X).P.folio.originElement := onePage_cell_unique _
1594   rw [hc, originImage_origin]
1595
1596 def dominionMap {X Y : DomIns} (f : X --> Y) :
1597   dominionAtlas X --> dominionAtlas Y where
1598   P := Unit onePagePag.E
1599   A :=

```

```

1600   { app := fun _ => f
1601     naturality := by intros; apply DomIns.hom_ext; intro; rfl }
1602
1603 theorem dominationMap_isTraversal {X Y : DomIns} (f : X --> Y) :
1604   IsTraversal (dominionMap f) := by
1605     refine <|Function.injective_id, fun _ _ h => h, ?_|>
1606     intro x t _
1607     exact atlasMap_all_covered (dominionAtlas_isMap Y) _ _
1608
1609 theorem dominationMap_isStable {X Y : DomIns} (f : X --> Y) :
1610   IsStableTraversal
1611     (X := WideSubcategory.mk (dominionAtlas X))
1612     (Y := WideSubcategory.mk (dominionAtlas Y))
1613     <|dominionMap f, dominationMap_isTraversal f|> := rfl
1614
1615 def DomInc : DomIns ==> StaAtlTrav where
1616   obj X := WideSubcategory.mk (WideSubcategory.mk (dominionAtlas X))
1617   map f := <|<|dominionMap f, dominationMap_isTraversal f|>,
1618     dominationMap_isStable f|>
1619   map_id _ := by
1620     apply Subtype.ext
1621     apply Subtype.ext
1622     apply AtlHom.ext
1623     . rfl
1624     . apply heq_of_eq
1625       ext
1626       rfl
1627   map_comp f g := by
1628     apply Subtype.ext
1629     apply Subtype.ext
1630     apply AtlHom.ext
1631     . rfl
1632     . apply heq_of_eq
1633       ext x t
1634       change (f >> g) t = (f >> g) t
1635     rfl
1636
1637 def onePageToAtlasP (Y : Atl) : onePagePag.E ==> Y.E :=
1638   (Functor.const onePagePag.E).obj Y.P.folio.originElement
1639
1640 theorem onePageToAtlasP_injective (Y : Atl) :
1641   Function.Injective (onePageToAtlasP Y).obj := by
1642     intro x y _
1643     exact onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
1644
1645 theorem onePage_elementLT_false (X : DomIns) (x y : (dominionAtlas X).E) :
1646   not elementLT (dominionAtlas X) x y := by
1647     intro h
1648     have hxy : x = y := onePage_cell_unique x |>.trans (onePage_cell_unique y).symm
1649     subst y
1650     let m := (dominionAtlas X).P.folio.commonBase x.1 x.1
1651     let l : LinearOrder ((dominionAtlas X).H.obj m) :=
1652       ((dominionAtlas X).P.folio.core.obj m.unop).linearOrder
1653     exact lt_irrefl _ h
1654
1655 /-- A stable traversal from a one-page atlas determines an embedding into
1656 the covered part of the target extent. -/
1657 def domIncToCoa {X : DomIns} {Y : StaAtlTrav} (f : DomInc.obj X --> Y) :
1658   X --> Coa.obj Y where
1659   toFun x := by
1660     have hc := f.1.2.2.2 onePageFolio.originElement x
1661     (atlasMap_all_covered (dominionAtlas_isMap X) _ x)
1662     exact coveredMap (X := Y.obj.obj) (eqToHom f.2)
1663     <|f.1.1.A.app onePageFolio.originElement x, hc|>
1664   inj' := fun a b h => by
1665     apply (f.1.1.A.app onePageFolio.originElement).injective
1666     apply (Y.obj.obj.G.map (eqToHom f.2)).injective
1667     exact congrArg Subtype.val h
1668
1669 /-- Conversely, an embedding into the coalition supplies the unique stable
1670 traversal from the corresponding one-page atlas. -/
1671 def coaToDomInc {X : DomIns} {Y : StaAtlTrav} (f : X --> Coa.obj Y) :
1672   DomInc.obj X --> Y := by

```

```

1673 let p : (dominionAtlas X).E ==> Y.obj.obj.P.E := onePageToAtlasP Y.obj.obj
1674 let a : (dominionAtlas X).G --> p then Y.obj.obj.G :=
1675   { app := fun _ =>
1676     { toFun := fun x => (f x).1
1677       inj' := fun x y h => f.injective (Subtype.ext h) }
1678     naturality := by
1679       intro x y g
1680       apply DomIns.hom_ext
1681       intro t
1682       have hxy : x = y := onePage_cell_unique x |>.trans
1683         (onePage_cell_unique y).symm
1684       subst y
1685       have hg : g = Id x := by
1686         apply CategoryOfElements.ext
1687         apply Subsingleton.elim
1688       subst g
1689       simp [p] }
1690 let h : dominionAtlas X --> Y.obj.obj := </p, a/>
1691 have htrav : IsTraversal h := by
1692   refine <|onePageToAtlasP_injective Y.obj.obj, ?_, ?_||>
1693   . intro x y hxy
1694   exact (onePage_elementLT_false X x y hxy).elim
1695   . intro x t _
1696   change Covered Y.obj.obj Y.obj.obj.P.folio.originElement (f t).1
1697   exact (f t).2
1698   exact <|<|h, htrav|>, rfl|>
1699
1700 def coaDomIncIso (X : DomIns) : Coa.obj (DomInc.obj X) Iso X where
1701   hom :=
1702     { toFun := fun x => x.1
1703       inj' := fun x y h => Subtype.ext h }
1704   inv :=
1705     { toFun := fun x => <|x, atlasMap_all_covered
1706       (dominionAtlas_isMap X) onePageFolio.originElement x|>
1707       inj' := fun x y h => congrArg Subtype.val h }
1708   hom_inv_id := by
1709     apply DomIns.hom_ext
1710     intro x
1711     apply Subtype.ext
1712     rfl
1713   inv_hom_id := by
1714     apply DomIns.hom_ext
1715     intro x
1716     rfl
1717
1718 /-- The hom-set equivalence expressing that one-page insertion is left
1719 adjoint to coalization. -/
1720 def domIncCoaHomEquiv (X : DomIns) (Y : StaAtlTrav) :
1721   (DomInc.obj X --> Y) Equiv (X --> Coa.obj Y) where
1722   toFun := domIncToCoa
1723   invFun := coaToDomInc
1724   left_inv := by
1725     intro f
1726     apply Subtype.ext
1727     apply Subtype.ext
1728     have hP : (coaToDomInc (domIncToCoa f)).1.1.P = f.1.1.P := by
1729       exact CategoryTheory.Functor.ext
1730         (fun x => f.2.symm.trans
1731           (congrArg f.1.1.P.obj (onePage_cell_unique x).symm))
1732       (fun _ _ => by
1733         apply CategoryOfElements.ext
1734         apply Subsingleton.elim)
1735     apply AtlHom.ext _ _ hP
1736     apply NatTrans.hext_right _ _
1737     (congrArg (fun Q => Q then Y.obj.obj.G) hP)
1738     intro x
1739     dsimp [coaToDomInc, domIncToCoa, onePageToAtlasP]
1740     have hx : x = onePageFolio.originElement := onePage_cell_unique x
1741     subst x
1742     let e := f.1.1.A.app onePageFolio.originElement
1743     have hrec :
1744       { toFun := fun x =>
1745         (coveredMap (X := Y.obj.obj) (eqToHom f.2)

```

```

1746         <|e x, by
1747             exact f.1.2.2.2 onePageFolio.originElement x
1748             (atlasMap_all_covered (dominionAtlas_isMap X) _ x)|>).1
1749     inj' := by
1750         intro a b hab
1751         apply e.injective
1752         apply (Y.obj.obj.G.map (eqToHom f.2)).injective
1753         exact hab } = e >> Y.obj.obj.G.map (eqToHom f.2) := by
1754     apply DomIns.hom_ext
1755     intro t
1756     rfl
1757     exact (heq_of_eq hrec).trans
1758         (embedding_comp_map_eqToHom_heq Y.obj.obj.G f.2 e)
1759 right_inv := by
1760     intro f
1761     apply DomIns.hom_ext
1762     intro x
1763     apply Subtype.ext
1764     change Y.obj.obj.G.map (Id Y.obj.obj.P.folio.originElement) (f x).1 = (f x).1
1765     exact congrFun (congrArg Function.Embedding.toFun
1766         (Y.obj.obj.G.map_id Y.obj.obj.P.folio.originElement)) (f x).1
1767
1768 theorem domIncCoaHomEquiv_naturality_left {X X' : DomIns} (f : X' --> X)
1769 {Y : StaAtlTrav} (g : DomInc.obj X --> Y) :
1770     domIncCoaHomEquiv X' Y (DomInc.map f >> g) =
1771     f >> domIncCoaHomEquiv X Y g := by
1772     apply DomIns.hom_ext
1773     intro x
1774     apply Subtype.ext
1775     rfl
1776
1777 theorem domIncCoaHomEquiv_naturality_right {X : DomIns} {Y Y' : StaAtlTrav}
1778 (f : DomInc.obj X --> Y) (g : Y --> Y') :
1779     domIncCoaHomEquiv X Y' (f >> g) =
1780     domIncCoaHomEquiv X Y f >> Coa.map g := by
1781     apply DomIns.hom_ext
1782     intro x
1783     apply Subtype.ext
1784     let t : Coa.obj (DomInc.obj X) :=
1785         <|x, atlasMap_all_covered
1786             (dominionAtlas_isMap X) onePageFolio.originElement x|>
1787     change (Coa.map (f >> g) t).1 = (Coa.map g (Coa.map f t)).1
1788     exact congrArg Subtype.val <| congrFun
1789         (congrArg Function.Embedding.toFun (Coa.map_comp f g)) t
1790
1791 /-- The unconditional adjunction promised by the Domanial Inclusion
1792 construction. -/
1793 def dominionAdjunction : DomInc -| Coa :=
1794     Adjunction.mkOfHomEquiv
1795     { homEquiv := domIncCoaHomEquiv
1796       homEquiv_naturality_left_symm := by
1797         intro X' X Y f g
1798         apply (domIncCoaHomEquiv X' Y).injective
1799         simpa using
1800             (domIncCoaHomEquiv_naturality_left f
1801             ((domIncCoaHomEquiv X Y).symm g)).symm
1802       homEquiv_naturality_right := by
1803         intro X Y Y' f g
1804         exact domIncCoaHomEquiv_naturality_right f g }
1805
1806 theorem dominionLemma : Nonempty (DomInc -| Coa) :=
1807     <|dominionAdjunction|>
1808
1809 abbrev DaTra := Atlop ==> Type
1810
1811 def Yo : Atl ==> DaTra := yoneda
1812
1813 /-- A concrete certificate of the statement that `DaTra` is the displayed
1814 presheaf category. -/
1815 def datraToposPresentation : DaTra EquivCat (Atlop ==> Type) :=
1816     CategoryTheory.Equivalence.refl
1817
1818 structure Navigation (D : DaTra) where

```

```

1819 A : Atl
1820 hom : Yo.obj A --> D
1821 mono : Mono hom
1822
1823 attribute [instance] Navigation.mono
1824
1825 structure Expedition (D : DaTra) extends Navigation D where
1826   atlasMap : IsAtlasMap A
1827
1828 abbrev DaTraRestriction (W : Type*) [Category W] := Wop ==> Type
1829
1830 abbrev DaTrap := DaTraRestriction AtlTrap
1831 abbrev DaTrav := DaTraRestriction AtlTrav
1832
1833 /-- Presheaves on the wide category of atlas traversals. -/
1834 abbrev AtlTravPSh := AtlTravop ==> Type
1835
1836 /-- Forget the action of an atlas presheaf on non-traversal arrows. -/
1837 def DaTra.restrictToAtlTrav : DaTra ==> AtlTravPSh :=
1838   (Functor.whiskeringLeft AtlTravop Atlop Type).obj AtlTravInc.op
1839
1840 /-- The promised identification is definitional after restricting the
1841 indexing category. -/
1842 def DaTrav.atlTravPShEquiv : DaTrav EquivCat AtlTravPSh :=
1843   CategoryTheory.Equivalence.refl
1844
1845 /-- Presheaves on atlas federations of stable atlas traversals. -/
1846 abbrev DaTratPresheaf := StableAtlasFamilyop ==> Type 3
1847
1848 /-- The all-objects wrapper gives Day convolution its own monoidal structure,
1849 separate from the pointwise cartesian structure on a raw functor category. -/
1850 def IsStableDataTransformation : ObjectProperty DaTratPresheaf := fun _ => True
1851
1852 /-- Stable data transformations: presheaves whose indexing arrows are stable atlas
1853 traversals. Stability is therefore enforced by the source category. -/
1854 abbrev DaTrat := IsStableDataTransformation.FullSubcategory
1855
1856 abbrev DaTratInc : DaTrat ==> DaTratPresheaf := IsStableDataTransformation.iota
1857
1858 def IsDaTraMap : ObjectProperty DaTra := fun D =>
1859   forall nav : Navigation D, IsAtlasMap nav.A
1860
1861 abbrev DaTraMap := IsDaTraMap.FullSubcategory
1862
1863 def AtlI : Atl := dominionAtlas emptyDominion
1864
1865 theorem AtlI_eq_domInc_zero :
1866   AtlI = (DomInc.obj emptyDominion).obj.obj := rfl
1867
1868 def Folio.pageValue (W : Folio) (m : Fin W.length) :
1869   (W.core.obj m).unop.Obj := by
1870   letI : NeZero W.length := <|Nat.ne_of_gt W.positive|>
1871   let q : (W.core.obj m).unop --> (W.core.obj W.originIndex).unop :=
1872     (W.core.map (homOfLE (Fin.zero_le m))).unop
1873   exact Classical.choose (q.point_surjective W.originValue)
1874
1875 theorem Folio.map_unop_comp_apply (W : Folio) {i j k : Fin W.length}
1876   (hik : i --> k) (hij : i --> j) (hjk : j --> k)
1877   (z : (W.core.obj k).unop.Obj) :
1878   (W.core.map hik).unop z =
1879     (W.core.map hij).unop ((W.core.map hjk).unop z) := by
1880   have e : hik = hij >> hjk := Subsingleton.elim _ _
1881   subst hik
1882   have h := congrArg Quiver.Hom.unop (W.core.map_comp hij hjk)
1883   exact congrFun (congrArg ConHom.toFun h) z
1884
1885 def bouquetLength (X Y : Folio) : Nat := max X.length Y.length + 1
1886
1887 def bouquetDepth (X Y : Folio) : Nat := max X.length Y.length
1888
1889 theorem bouquetLength_pos (X Y : Folio) : 0 < bouquetLength X Y := by
1890   simp [bouquetLength]
1891

```



```

1892 instance (X Y : Folio) : NeZero (bouquetLength X Y) :=
1893   <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
1894
1895 def bouquetChain (X Y : Folio) : Fin (bouquetLength X Y) → Chain :=
1896   Fin.cases singletonChain (fun m : Fin (bouquetDepth X Y) =>
1897     Chain.sum (X.core.obj (X.paddedIndex m.1)).unop
1898       (Y.core.obj (Y.paddedIndex m.1)).unop)
1899
1900 def bouquetValue (X Y : Folio) (m : Fin (bouquetLength X Y)) :
1901   (bouquetChain X Y m).Obj :=
1902   Fin.cases () (fun n => toLex (Sum.inl (X.pageValue (X.paddedIndex n.1)))) m
1903
1904 def bouquetConMap (X Y : Folio) {i j : Fin (bouquetLength X Y)} (h : i ≤ j) :
1905   bouquetChain X Y j --> bouquetChain X Y i := by
1906   revert j
1907   refine Fin.cases ?_ (fun i => ?_) i
1908   . rename_i j
1909   intro h
1910   refine
1911     { toFun := fun _ => ()
1912       monotone := fun _ _ => le_rfl
1913       point_surjective := fun z => by
1914         refine <|bouquetValue X Y j, ?_|>
1915         change (()) : Unit = z
1916         exact Subsingleton.elim _ _ }
1917   . rename_i j
1918   intro h
1919   revert h
1920   refine Fin.cases (by
1921     intro h
1922     change i.1 + 1 ≤ 0 at h
1923     omega) (fun j => ?_) j
1924   intro h
1925   have hij : i ≤ j := Fin.succ_le_succ_iff.mp h
1926   let hx : X.paddedIndex i.1 ≤ X.paddedIndex j.1 := X.paddedIndex_mono hij
1927   let hy : Y.paddedIndex i.1 ≤ Y.paddedIndex j.1 := Y.paddedIndex_mono hij
1928   exact ConHom.sum (X.core.map (homOfLE hx)).unop
1929     (Y.core.map (homOfLE hy)).unop
1930
1931 def bouquetFolio (X Y : Folio) : Folio where
1932   length := bouquetLength X Y
1933   positive := bouquetLength_pos X Y
1934   core :=
1935     { obj := fun m => op (bouquetChain X Y m)
1936       map := fun f => (bouquetConMap X Y (leOfHom f)).op
1937       map_id := by
1938         intro m
1939         refine Fin.cases ?_ (fun m => ?_) m
1940         . apply Quiver.Hom.unop_inj
1941         apply ConHom.ext
1942         intro z
1943         change (()) : Unit = ()
1944         rfl
1945       apply Quiver.Hom.unop_inj
1946       apply ConHom.ext
1947       intro w
1948       generalize hz : ofLex w = z
1949       rcases z with z | z
1950       . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simp using hz)
1951         subst w
1952         simp [bouquetConMap, ConHom.sum]
1953         change toLex (Sum.inl z) = toLex (Sum.inl z)
1954         rfl
1955       . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simp using hz)
1956         subst w
1957         simp [bouquetConMap, ConHom.sum]
1958         change toLex (Sum.inr z) = toLex (Sum.inr z)
1959         rfl
1960     map_comp := by
1961       intro i j k f g
1962       let I : NeZero (bouquetLength X Y) := <|Nat.ne_of_gt (bouquetLength_pos X Y)|>
1963       revert j k
1964       refine Fin.cases ?_ (fun i => ?_) i

```

```

1965     . rename_i jTarget kTarget
1966     intro f g
1967     apply Quiver.Hom.unop_inj
1968     apply ConHom.ext
1969     intro z
1970     change ((() : Unit) = ())
1971     rfl
1972   . rename_i iOrig jTarget kTarget
1973   intro f g
1974   have hj : jTarget != 0 := by
1975     intro e
1976     have hf := leOfHom f
1977     rw [e] at hf
1978     change i.1 + 1 <= 0 at hf
1979     omega
1980   obtain <|j, rfl|> := Fin.eq_succ_of_ne_zero hj
1981   have hk : kTarget != 0 := by
1982     intro e
1983     have hg := leOfHom g
1984     rw [e] at hg
1985     change j.1 + 1 <= 0 at hg
1986     omega
1987   obtain <|k, rfl|> := Fin.eq_succ_of_ne_zero hk
1988   apply Quiver.Hom.unop_inj
1989   apply ConHom.ext
1990   intro w
1991   generalize hz : ofLex w = z
1992   rcases z with z | z
1993   . have hz' : w = toLex (Sum.inl z) := ofLex.injective (by simpa using hz)
1994     subst w
1995     simpa only [bouquetConMap, ConHom.sum] using congrArg
1996       (fun q => toLex (Sum.inl q)) (X.map_unop_comp_apply _ _ z)
1997   . have hz' : w = toLex (Sum.inr z) := ofLex.injective (by simpa using hz)
1998     subst w
1999     simpa only [bouquetConMap, ConHom.sum] using congrArg
2000       (fun q => toLex (Sum.inr q)) (Y.map_unop_comp_apply _ _ z) }
2001   originEquiv := by
2002     exact singletonObjEquiv
2003
2004   def domSum (X Y : DomIns) : DomIns where
2005     toDom :=
2006       { Carrier := Sum X Y
2007       rank :=
2008         { toFun := fun z => match z with
2009           | .inl x => 2 * X.toDom.rank x
2010           | .inr y => 2 * Y.toDom.rank y + 1
2011         inj' := by
2012           intro a b h
2013           rcases a with a | a <;> rcases b with b | b
2014           . simp only at h
2015           exact congrArg Sum.inl (X.toDom.rank.injective (by omega))
2016           . simp only at h
2017             omega
2018           . simp only at h
2019             omega
2020           . simp only at h
2021             exact congrArg Sum.inr (Y.toDom.rank.injective (by omega)) } }
2022
2023   def domSum.inl (X Y : DomIns) : X --> domSum X Y where
2024     toFun := Sum.inl
2025     inj' := Sum.inl_injective
2026
2027   def domSum.inr (X Y : DomIns) : Y --> domSum X Y where
2028     toFun := Sum.inr
2029     inj' := Sum.inr_injective
2030
2031   def domSum.map {X_1 X_2 Y_1 Y_2 : DomIns} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2032     domSum X_1 X_2 --> domSum Y_1 Y_2 where
2033     toFun := Sum.map f g
2034     inj' := by
2035       intro a b h
2036       rcases a with a | a <;> rcases b with b | b
2037       . exact congrArg Sum.inl (f.injective (Sum.inl.inj h))

```

```

2038 . exact Sum.noConfusion h
2039 . exact Sum.noConfusion h
2040 . exact congrArg Sum.inr (g.injective (Sum.inr.inj h))
2041
2042 def tallMergePage (X Y : TallAtlas) : Nat → Type
2043 | 0 => Unit
2044 | n + 1 => Sum (X.H.obj (op n)) (Y.H.obj (op n))
2045
2046 def tallMergePageMap (X Y : TallAtlas) {i j : Nat} (h : j ≤ i) :
2047   tallMergePage X Y i → tallMergePage X Y j := by
2048   induction j with
2049   | zero => exact fun _ => ()
2050   | succ j =>
2051     induction i with
2052     | zero => omega
2053     | succ i =>
2054       exact Sum.map
2055         (X.H.map (homOfLE (Nat.succ_le_succ_iff.mp h)).op)
2056         (Y.H.map (homOfLE (Nat.succ_le_succ_iff.mp h)).op)
2057
2058 def tallMergeH (X Y : TallAtlas) : Natop ==> Type where
2059   obj n := tallMergePage X Y n.unop
2060   map f := tallMergePageMap X Y (leOfHom f.unop)
2061   map_id n := by
2062     funext z
2063     rcases n with <|n|>
2064     cases n with
2065     | zero => rfl
2066     | succ n =>
2067       rcases z with z | z
2068       . apply congrArg Sum.inl
2069       simp
2070       . apply congrArg Sum.inr
2071       simp
2072   map_comp := by
2073     intro ni nj nk f g
2074     funext z
2075     rcases ni with <|i|>
2076     rcases nj with <|j|>
2077     rcases nk with <|k|>
2078     cases k with
2079     | zero => rfl
2080     | succ k =>
2081       have hj : j ≠ 0 := by
2082         intro e
2083         subst j
2084         have hg := leOfHom g.unop
2085         change k + 1 ≤ 0 at hg
2086         omega
2087       obtain <|j, rfl|> := Nat.exists_eq_succ_of_ne_zero hj
2088       have hi : i ≠ 0 := by
2089         intro e
2090         subst i
2091         have hf := leOfHom f.unop
2092         change j + 1 ≤ 0 at hf
2093         omega
2094       obtain <|i, rfl|> := Nat.exists_eq_succ_of_ne_zero hi
2095       let fij : (op i : Natop) → op j :=
2096         (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom f.unop))).op
2097       let fjk : (op j : Natop) → op k :=
2098         (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom g.unop))).op
2099       let fik : (op i : Natop) → op k :=
2100         (homOfLE (Nat.succ_le_succ_iff.mp (leOfHom (f >> g).unop))).op
2101       have hcomp : fik = fij >> fjk := Subsingleton.elim _ _
2102       rcases z with z | z
2103       . apply congrArg Sum.inl
2104       change X.H.map fik z = X.H.map fjk (X.H.map fij z)
2105       rw [hcomp, X.H.map_comp]
2106       rfl
2107       . apply congrArg Sum.inr
2108       change Y.H.map fik z = Y.H.map fjk (Y.H.map fij z)
2109       rw [hcomp, Y.H.map_comp]
2110       rfl

```

```

2111
2112 def bouquetPag (X Y : Atl) : Pag := <|bouquetFolio X.P.folio Y.P.folio|>
2113
2114 def bouquetLeftIndex (X Y : Atl) (m : Fin X.P.folio.length) :
2115   Fin (bouquetLength X.P.folio Y.P.folio) :=
2116   <|m.1 + 1, by simp [bouquetLength]|>
2117
2118 def bouquetRightIndex (X Y : Atl) (m : Fin Y.P.folio.length) :
2119   Fin (bouquetLength X.P.folio Y.P.folio) :=
2120   <|m.1 + 1, by simp [bouquetLength]|>
2121
2122 def bouquetLeftPred (X Y : Atl) (m : Fin X.P.folio.length) :
2123   Fin (bouquetDepth X.P.folio Y.P.folio) :=
2124   <|m.1, lt_of_lt_of_le m.2 (le_max_left _ _)|>
2125
2126 def bouquetRightPred (X Y : Atl) (m : Fin Y.P.folio.length) :
2127   Fin (bouquetDepth X.P.folio Y.P.folio) :=
2128   <|m.1, lt_of_lt_of_le m.2 (le_max_right _ _)|>
2129
2130 @[simp]
2131 theorem bouquetLeftIndex_eq_succ (X Y : Atl) (m : Fin X.P.folio.length) :
2132   bouquetLeftIndex X Y m = (bouquetLeftPred X Y m).succ := rfl
2133
2134 @[simp]
2135 theorem bouquetRightIndex_eq_succ (X Y : Atl) (m : Fin Y.P.folio.length) :
2136   bouquetRightIndex X Y m = (bouquetRightPred X Y m).succ := rfl
2137
2138 def bouquetLeftElement (X Y : Atl) (x : X.E) : (bouquetPag X Y).E := by
2139   refine <|op (bouquetLeftPred X Y x.1.unop).succ, ?_|>
2140   change (bouquetChain X.P.folio Y.P.folio
2141     (bouquetLeftPred X Y x.1.unop).succ).Obj
2142   change Lex (Sum
2143     ((X.P.folio.core.obj (X.P.folio.paddedIndex x.1.unop.1)).unop.Obj)
2144     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex x.1.unop.1)).unop.Obj))
2145   exact toLex (Sum.inl (X.P.H.map
2146     (eqToHom (congrArg op (X.P.folio.paddedIndex_fin x.1.unop).symm)) x.2))
2147
2148 def bouquetRightElement (X Y : Atl) (y : Y.E) : (bouquetPag X Y).E := by
2149   refine <|op (bouquetRightPred X Y y.1.unop).succ, ?_|>
2150   change (bouquetChain X.P.folio Y.P.folio
2151     (bouquetRightPred X Y y.1.unop).succ).Obj
2152   change Lex (Sum
2153     ((X.P.folio.core.obj (X.P.folio.paddedIndex y.1.unop.1)).unop.Obj)
2154     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex y.1.unop.1)).unop.Obj))
2155   exact toLex (Sum.inr (Y.P.H.map
2156     (eqToHom (congrArg op (Y.P.folio.paddedIndex_fin y.1.unop).symm)) y.2))
2157
2158 def bouquetLeftMapHom (X Y : Atl) {x y : X.E} (q : x --> y) :
2159   bouquetLeftElement X Y x --> bouquetLeftElement X Y y := by
2160   rcases x with <|<|mx|>, kx|>
2161   rcases y with <|<|my|>, ky|>
2162   let hxy : my <= mx := leOfHom (Quiver.Hom.unop q.val)
2163   let h : bouquetLeftPred X Y my <= bouquetLeftPred X Y mx := by
2164     change my.1 <= mx.1
2165     exact hxy
2166   refine CategoryOfElements.homMk _ _
2167     (homOfLE (Fin.succ_le_succ_iff.mpr h)).op ?_
2168   dsimp only [bouquetLeftElement]
2169   simp only [bouquetLeftPred, bouquetPag, bouquetFolio, Folio.H,
2170     bouquetChain, bouquetConMap, Fin.cases_succ, ConHom.sum, Functor.leftOp_map,
2171     Functor.comp_obj, Functor.comp_map, Quiver.Hom.unop_op, Tra]
2172   change toLex (Sum.inl _) = toLex (Sum.inl _)
2173   congr 2
2174   have hqv : X.P.H.map q.val kx = ky := q.property
2175   rw [<- hqv]
2176   let hp : X.P.folio.paddedIndex my.1 <= X.P.folio.paddedIndex mx.1 :=
2177     X.P.folio.paddedIndex_mono hxy
2178   change X.P.H.map (homOfLE hp).op
2179     (X.P.H.map (eqToHom _) kx) = X.P.H.map (eqToHom _)
2180     (X.P.H.map q.val kx)
2181   rw [<- FunctorToTypes.map_comp_apply]
2182   rw [<- FunctorToTypes.map_comp_apply]
2183   congr 1

```

```

2184
2185 def bouquetRightMapHom (X Y : At1) {x y : Y.E} (q : x --> y) :
2186   bouquetRightElement X Y x --> bouquetRightElement X Y y := by
2187   rcases x with <|<|mx|>, kx|>
2188   rcases y with <|<|my|>, ky|>
2189   let hxy : my <= mx := leOfHom (Quiver.Hom.unop q.val)
2190   let h : bouquetRightPred X Y my <= bouquetRightPred X Y mx := by
2191     change my.1 <= mx.1
2192     exact hxy
2193   refine CategoryOfElements.homMk _ _
2194     (homOfLE (Fin.succ_le_succ_iff.mpr h)).op ?_
2195   dsimp only [bouquetRightElement]
2196   simp only [bouquetRightPred, bouquetPag, bouquetFolio, Folio.H,
2197     bouquetChain, bouquetConMap, Fin.cases_succ, ConHom.sum, Functor.leftOp_map,
2198     Functor.comp_obj, Functor.comp_map, Quiver.Hom.unop_op, Tra]
2199   change toLex (Sum.inr _) = toLex (Sum.inr _)
2200   congr 2
2201   have hqv : Y.P.H.map q.val kx = ky := q.property
2202   rw [<- hqv]
2203   let hp : Y.P.folio.paddedIndex my.1 <= Y.P.folio.paddedIndex mx.1 :=
2204     Y.P.folio.paddedIndex_mono hxy
2205   change Y.P.H.map (homOfLE hp).op
2206     (Y.P.H.map (eqToHom _) kx) = Y.P.H.map (eqToHom _)
2207     (Y.P.H.map q.val kx)
2208   rw [<- FunctorToTypes.map_comp_apply]
2209   rw [<- FunctorToTypes.map_comp_apply]
2210   congr 1
2211
2212 def bouquetLeftElements (X Y : At1) : X.E ==> (bouquetPag X Y).E where
2213   obj := bouquetLeftElement X Y
2214   map := bouquetLeftMapHom X Y
2215   map_id _ := by
2216     apply CategoryOfElements.ext
2217     apply Subsingleton.elim
2218   map_comp _ _ := by
2219     apply CategoryOfElements.ext
2220     apply Subsingleton.elim
2221
2222 def bouquetRightElements (X Y : At1) : Y.E ==> (bouquetPag X Y).E where
2223   obj := bouquetRightElement X Y
2224   map := bouquetRightMapHom X Y
2225   map_id _ := by
2226     apply CategoryOfElements.ext
2227     apply Subsingleton.elim
2228   map_comp _ _ := by
2229     apply CategoryOfElements.ext
2230     apply Subsingleton.elim
2231
2232 def imageDom {A R : DomIns} (f : A --> R) : DomIns where
2233   toDom :=
2234     { Carrier := Set.range f
2235       rank :=
2236         ({ toFun := Subtype.val
2237           inj' := Subtype.val_injective } : Set.range f --> R).trans R.toDom.rank }
2238
2239 def imageDom.inclusion {A R : DomIns} (f : A --> R) : imageDom f --> R where
2240   toFun := Subtype.val
2241   inj' := Subtype.val_injective
2242
2243 def imageDom.map {A B R : DomIns} (f : A --> R) (g : B --> R)
2244   (h : Set.range f subset Set.range g) : imageDom f --> imageDom g where
2245   toFun z := <|z.1, h z.2|>
2246   inj' := by
2247     intro a b e
2248     apply Subtype.ext
2249     exact congrArg (fun q => q.1) e
2250
2251 def imageDom.mapAcross {A B R S : DomIns} (f : A --> R) (g : B --> S)
2252   (h : R --> S) (factor : forall a, exists b, h (f a) = g b) :
2253   imageDom f --> imageDom g where
2254   toFun z := <|h z.1, by
2255     rcases z.2 with <|a, ha|>
2256     rcases factor a with <|b, hb|>

```

```

2257     exact <|b, (congrArg h ha.symm |>.trans hb).symm|>|>
2258   inj' := by
2259     intro a b e
2260     apply Subtype.ext
2261     apply h.injective
2262     exact congrArg Subtype.val e
2263
2264   def tallMergeRawDom (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) : DomIns := by
2265     rcases x with <|<|n|>, v|>
2266     cases n with
2267     | zero => exact domSum X.extent Y.extent
2268     | succ n =>
2269       exact match v with
2270       | .inl k => X.G.obj <|op n, k|>
2271       | .inr k => Y.G.obj <|op n, k|>
2272
2273   def tallMergeRootMap (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) :
2274     tallMergeRawDom X Y x --> domSum X.extent Y.extent := by
2275     rcases x with <|<|n|>, v|>
2276     cases n with
2277     | zero => exact Id _
2278     | succ n =>
2279       exact match v with
2280       | .inl k => X.G.map (X.toOrigin <|op n, k|>) >>
2281         domSum.inl X.extent Y.extent
2282       | .inr k => Y.G.map (Y.toOrigin <|op n, k|>) >>
2283         domSum.inr X.extent Y.extent
2284
2285   def tallMergeDom (X Y : TallAtlas) (x : (tallMergeH X Y).Elements) : DomIns :=
2286     imageDom (tallMergeRootMap X Y x)
2287
2288   theorem tallMergeRange_mono (X Y : TallAtlas)
2289     {x y : (tallMergeH X Y).Elements} (f : x --> y) :
2290     Set.range (tallMergeRootMap X Y x) subset
2291     Set.range (tallMergeRootMap X Y y) := by
2292     intro z hz
2293     rcases x with <|<|mx|>, vx|>
2294     rcases y with <|<|my|>, vy|>
2295     rcases hz with <|a, rfl|>
2296     cases mx with
2297     | zero =>
2298       have hmy : my = 0 := by
2299         have hf := leOfHom f.val.unop
2300         change my <= 0 at hf
2301         omega
2302       subst my
2303       exact <|a, rfl|>
2304     | succ n =>
2305       cases my with
2306       | zero => exact <|tallMergeRootMap X Y <|op n.succ, vx|> a, rfl|>
2307       | succ p =>
2308         have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom f.val.unop)
2309         rcases vx with k | k
2310         . have hfv : Sum.inl (X.H.map (homOfLE hpn).op k) = vy := by
2311           simp [tallMergeH, tallMergePageMap] using f.property
2312           subst vy
2313         let q := CategoryOfElements.homMk
2314           (<|op n, k|> : X.E)
2315           (<|op p, X.H.map (homOfLE hpn).op k|> : X.E)
2316           (homOfLE hpn).op rfl
2317         refine <|X.G.map q a, ?_|>
2318         change Sum.inl (X.G.map (X.toOrigin _) (X.G.map q a)) =
2319           Sum.inl (X.G.map (X.toOrigin _) a)
2320         apply congrArg Sum.inl
2321         have hc : q >> X.toOrigin _ = X.toOrigin _ :=
2322           CategoryOfElements.ext X.H _ (Subsingleton.elim _ _)
2323         have hm := X.G.map_comp q (X.toOrigin _)
2324         rw [hc] at hm
2325         exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
2326       . have hfv : Sum.inr (Y.H.map (homOfLE hpn).op k) = vy := by
2327         simp [tallMergeH, tallMergePageMap] using f.property
2328         subst vy
2329         let q := CategoryOfElements.homMk

```

```

2330         (<|op n, k|> : Y.E)
2331         (<|op p, Y.H.map (homOfLE hpn).op k|> : Y.E)
2332         (homOfLE hpn).op rfl
2333     refine <|Y.G.map q a, ?_|>
2334     change Sum.inr (Y.G.map (Y.toOrigin _) (Y.G.map q a)) =
2335         Sum.inr (Y.G.map (Y.toOrigin _) a)
2336     apply congrArg Sum.inr
2337     have hc : q >> Y.toOrigin _ = Y.toOrigin _ :=
2338         CategoryOfElements.ext Y.H _ _ (Subsingleton.elim _ _)
2339     have hm := Y.G.map_comp q (Y.toOrigin _)
2340     rw [hc] at hm
2341     exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
2342
2343 def tallMergeImageMap (X Y : TallAtlas)
2344   {x y : (tallMergeH X Y).Elements} (f : x --> y) :
2345   tallMergeDom X Y x --> tallMergeDom X Y y :=
2346   imageDom.map _ _ (tallMergeRange_mono X Y f)
2347
2348 def tallMergeG (X Y : TallAtlas) : (tallMergeH X Y).Elements ==> DomIns where
2349   obj := tallMergeDom X Y
2350   map := tallMergeImageMap X Y
2351   map_id _ := by
2352     apply DomIns.hom_ext
2353     intro z
2354     apply Subtype.ext
2355     rfl
2356   map_comp _ _ := by
2357     apply DomIns.hom_ext
2358     intro z
2359     apply Subtype.ext
2360     rfl
2361
2362 def tallMergeCellLT (X Y : TallAtlas)
2363   (x y : (tallMergeH X Y).Elements) : Prop := by
2364   rcases x with <|<|nx|>, vx|>
2365   rcases y with <|<|ny|>, vy|>
2366   cases nx with
2367   | zero => exact False
2368   | succ nx =>
2369     cases ny with
2370     | zero => exact False
2371     | succ ny =>
2372       exact match vx, vy with
2373       | .inl k, .inl l => X.cellLT <|op nx, k|> <|op ny, l|>
2374       | .inl _, .inr _ => True
2375       | .inr _, .inl _ => False
2376       | .inr k, .inr l => Y.cellLT <|op nx, k|> <|op ny, l|>
2377
2378 def tallMerge (X Y : TallAtlas) : TallAtlas where
2379   H := tallMergeH X Y
2380   originValue := ()
2381   originUnique x := by
2382     change Unit at x
2383     cases x
2384     rfl
2385   G := tallMergeG X Y
2386   cellLT := tallMergeCellLT X Y
2387   coveredExtent t := match t.1 with
2388   | .inl a => X.coveredExtent a
2389   | .inr b => Y.coveredExtent b
2390
2391 def tallMergeLeftObj (X Y : TallAtlas) (x : X.E) : (tallMerge X Y).E :=
2392   <|op (x.1.unop + 1), Sum.inl x.2|>
2393
2394 def tallMergeRightObj (X Y : TallAtlas) (y : Y.E) : (tallMerge X Y).E :=
2395   <|op (y.1.unop + 1), Sum.inr y.2|>
2396
2397 def tallMergeLeftMap (X Y : TallAtlas) {x y : X.E} (f : x --> y) :
2398   tallMergeLeftObj X Y x --> tallMergeLeftObj X Y y := by
2399   rcases x with <|<|n|>, kx|>
2400   rcases y with <|<|m|>, ky|>
2401   have hmn : m <= n := leOfHom f.val.unop
2402   refine CategoryOfElements.homMk _ _

```

```

2403     (homOfLE (Nat.succ_le_succ hmn)).op ?_
2404 change Sum.inl (X.H.map (homOfLE hmn).op kx) = Sum.inl ky
2405 congr 1
2406 have ef : f.val = (homOfLE hmn).op := Subsingleton.elim _ _
2407 rw [← ef]
2408 exact f.property
2409
2410 def tallMergeRightMap (X Y : TallAtlas) {x y : Y.E} (f : x --> y) :
2411   tallMergeRightObj X Y x --> tallMergeRightObj X Y y := by
2412   rcases x with <|<n|>, kx|>
2413   rcases y with <|<m|>, ky|>
2414   have hmn : m <= n := leOfHom f.val.unop
2415   refine CategoryOfElements.homMk _ _
2416     (homOfLE (Nat.succ_le_succ hmn)).op ?_
2417 change Sum.inr (Y.H.map (homOfLE hmn).op kx) = Sum.inr ky
2418 congr 1
2419 have ef : f.val = (homOfLE hmn).op := Subsingleton.elim _ _
2420 rw [← ef]
2421 exact f.property
2422
2423 def tallMergeLeft (X Y : TallAtlas) : X.E ==> (tallMerge X Y).E where
2424   obj := tallMergeLeftObj X Y
2425   map := tallMergeLeftMap X Y
2426   map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2427   map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2428
2429 def tallMergeRight (X Y : TallAtlas) : Y.E ==> (tallMerge X Y).E where
2430   obj := tallMergeRightObj X Y
2431   map := tallMergeRightMap X Y
2432   map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2433   map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2434
2435 def tallHorPObj {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2436   (x : (tallMerge X_1 X_2).E) : (tallMerge Y_1 Y_2).E := by
2437   rcases x with <|<n|>, v|>
2438   cases n with
2439   | zero => exact (tallMerge Y_1 Y_2).originElement
2440   | succ n =>
2441     exact match v with
2442     | .inl k => tallMergeLeftObj Y_1 Y_2 (f.P.obj <|op n, k|>)
2443     | .inr k => tallMergeRightObj Y_1 Y_2 (g.P.obj <|op n, k|>)
2444
2445 def tallHorPHom {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2446   {x y : (tallMerge X_1 X_2).E} (q : x --> y) :
2447   tallHorPObj f g x --> tallHorPObj f g y := by
2448   rcases x with <|<mx|>, vx|>
2449   rcases y with <|<my|>, vy|>
2450   cases mx with
2451   | zero =>
2452     have hmy : my = 0 := by
2453       have hq := leOfHom q.val.unop
2454       change my <= 0 at hq
2455       omega
2456     subst my
2457     exact Id _
2458   | succ n =>
2459     cases my with
2460     | zero => exact (tallMerge Y_1 Y_2).toOrigin _
2461     | succ p =>
2462       have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom q.val.unop)
2463       rcases vx with k | k
2464       . have hqv : Sum.inl (X_1.H.map (homOfLE hpn).op k) = vy := by
2465         simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2466         subst vy
2467         let r := CategoryOfElements.homMk
2468           (<|op n, k|> : X_1.E)
2469           (<|op p, X_1.H.map (homOfLE hpn).op k|> : X_1.E)
2470           (homOfLE hpn).op rfl
2471         exact tallMergeLeftMap Y_1 Y_2 (f.P.map r)
2472       . have hqv : Sum.inr (X_2.H.map (homOfLE hpn).op k) = vy := by
2473         simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2474         subst vy
2475         let r := CategoryOfElements.homMk

```



```

2476         (<|op n, k|> : X_2.E)
2477         (<|op p, X_2.H.map (homOfLE hpn).op k|> : X_2.E)
2478         (homOfLE hpn).op rfl
2479         exact tallMergeRightMap Y_1 Y_2 (g.P.map r)
2480
2481 def tallHorP {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2482   (tallMerge X_1 X_2).E ==> (tallMerge Y_1 Y_2).E where
2483   obj := tallHorPObj f g
2484   map := tallHorPHom f g
2485   map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2486   map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2487
2488 def TallAtlas.originImage (X : TallAtlas) (x : X.E) (a : X.G.obj x) : X.extent :=
2489   X.G.map (X.toOrigin x) a
2490
2491 @[simp]
2492 theorem TallAtlas.originImage_origin (X : TallAtlas) (a : X.extent) :
2493   X.originImage X.originElement a = a := by
2494     change X.G.map (X.toOrigin X.originElement) a = a
2495     have h : X.toOrigin X.originElement = Id X.originElement :=
2496       Subsingleton.elim _ _
2497     rw [h, X.G.map_id]
2498     rfl
2499
2500 def TallAtlas.extentMap {X Y : TallAtlas} (f : X --> Y) : X.extent --> Y.extent :=
2501   f.A.app X.originElement >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))
2502
2503 @[simp]
2504 theorem TallAtlas.extentMap_id (X : TallAtlas) :
2505   TallAtlas.extentMap (Id X) = Id X.extent := by
2506     apply DomIns.hom_ext
2507     intro a
2508     change X.G.map (X.toOrigin X.originElement) a = a
2509     have h : X.toOrigin X.originElement = Id X.originElement := Subsingleton.elim _ _
2510     rw [h]
2511     rw [X.G.map_id]
2512     rfl
2513
2514 theorem TallAtlas.extentMap_originImage {X Y : TallAtlas} (f : X --> Y)
2515   (x : X.E) (a : X.G.obj x) :
2516   TallAtlas.extentMap f (X.originImage x a) =
2517     Y.originImage (f.P.obj x) (f.A.app x a) := by
2518     simp only [TallAtlas.extentMap, TallAtlas.originImage]
2519     have hn := f.A.naturality (X.toOrigin x)
2520     have hc : f.P.map (X.toOrigin x) >> Y.toOrigin (f.P.obj X.originElement) =
2521       Y.toOrigin (f.P.obj x) := by
2522       apply CategoryOfElements.ext
2523       apply Subsingleton.elim
2524     rw [hc, Y.G.map_comp]
2525     exact congrFun (congrArg Function.Embedding.toFun
2526       (congrArg (fun k => k >> Y.G.map (Y.toOrigin (f.P.obj X.originElement)))) hn) a
2527
2528 @[simp]
2529 theorem TallAtlas.extentMap_comp {X Y Z : TallAtlas} (f : X --> Y) (g : Y --> Z) :
2530   TallAtlas.extentMap (f >> g) =
2531     TallAtlas.extentMap f >> TallAtlas.extentMap g := by
2532     simp only [TallAtlas.extentMap]
2533     have hn := g.A.naturality (Y.toOrigin (f.P.obj X.originElement))
2534     simp only [Functor.comp_map] at hn
2535     have hc : g.P.map (Y.toOrigin (f.P.obj X.originElement)) >>
2536       Z.toOrigin (g.P.obj Y.originElement) =
2537       Z.toOrigin (g.P.obj (f.P.obj X.originElement)) := by
2538       apply CategoryOfElements.ext
2539       apply Subsingleton.elim
2540     change f.A.app X.originElement >> g.A.app (f.P.obj X.originElement) >>
2541       Z.G.map (Z.toOrigin (g.P.obj (f.P.obj X.originElement))) =
2542       (f.A.app X.originElement >> Y.G.map (Y.toOrigin (f.P.obj X.originElement))) >>
2543       g.A.app Y.originElement >> Z.G.map (Z.toOrigin (g.P.obj Y.originElement))
2544     simp only [Category.assoc]
2545     rw [Category.assoc (Y.G.map (Y.toOrigin (f.P.obj X.originElement)))]
2546     (g.A.app Y.originElement) (Z.G.map (Z.toOrigin (g.P.obj Y.originElement)))
2547     rw [hn]
2548     rw [Category.assoc (g.A.app (f.P.obj X.originElement))]

```

```

2549   rw [← Z.G.map_comp]
2550   rw [hc]
2551
2552   def tallHorExtentMap {X_1 X_2 Y_1 Y_2 : TallAtlas}
2553     (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2554     domSum X_1.extent X_2.extent --> domSum Y_1.extent Y_2.extent :=
2555     domSum.map (TallAtlas.extentMap f) (TallAtlas.extentMap g)
2556
2557   theorem tallHorRootFactor {X_1 X_2 Y_1 Y_2 : TallAtlas}
2558     (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2559     (x : (tallMerge X_1 X_2).E) (a : tallMergeRawDom X_1 X_2 x) :
2560     exists b : tallMergeRawDom Y_1 Y_2 (tallHorPObj f g x),
2561       tallHorExtentMap f g (tallMergeRootMap X_1 X_2 x a) =
2562       tallMergeRootMap Y_1 Y_2 (tallHorPObj f g x) b := by
2563   rcases x with <|<n|>, v|>
2564   cases n with
2565   | zero => exact <|tallHorExtentMap f g a, rfl|>
2566   | succ n =>
2567     rcases v with k | k
2568     . let x_0 : X_1.E := <|op n, k|>
2569       let y_0 := f.P.obj x_0
2570       refine <|f.A.app x_0 a, ?_|>
2571       change Sum.inl (TallAtlas.extentMap f (X_1.originImage x_0 a)) =
2572       Sum.inl (Y_1.originImage y_0 (f.A.app x_0 a))
2573       exact congrArg Sum.inl (TallAtlas.extentMap_originImage f x_0 a)
2574     . let x_0 : X_2.E := <|op n, k|>
2575       let y_0 := g.P.obj x_0
2576       refine <|g.A.app x_0 a, ?_|>
2577       change Sum.inr (TallAtlas.extentMap g (X_2.originImage x_0 a)) =
2578       Sum.inr (Y_2.originImage y_0 (g.A.app x_0 a))
2579       exact congrArg Sum.inr (TallAtlas.extentMap_originImage g x_0 a)
2580
2581   def tallHorA {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2582     (tallMerge X_1 X_2).G --> tallHorP f g then (tallMerge Y_1 Y_2).G where
2583   app x := imageDom.mapAcross
2584     (tallMergeRootMap X_1 X_2 x)
2585     (tallMergeRootMap Y_1 Y_2 (tallHorPObj f g x))
2586     (tallHorExtentMap f g) (tallHorRootFactor f g x)
2587   naturality := by
2588     intro x y q
2589     apply DomIns.hom_ext
2590     intro z
2591     apply Subtype.ext
2592     rfl
2593
2594   def tallHorMap {X_1 X_2 Y_1 Y_2 : TallAtlas} (f : X_1 --> Y_1) (g : X_2 --> Y_2) :
2595     tallMerge X_1 X_2 --> tallMerge Y_1 Y_2 where
2596   P := tallHorP f g
2597   A := tallHorA f g
2598
2599   @[simp]
2600   theorem tallHorMap_extentMap_val {X_1 X_2 Y_1 Y_2 : TallAtlas}
2601     (f : X_1 --> Y_1) (g : X_2 --> Y_2)
2602     (z : (tallMerge X_1 X_2).extent) :
2603     (TallAtlas.extentMap (tallHorMap f g) z).1 =
2604     tallHorExtentMap f g z.1 := by
2605   rfl
2606
2607   theorem tallHorMap_id (X Y : TallAtlas) :
2608     tallHorMap (Id X) (Id Y) = Id (tallMerge X Y) := by
2609   have hP : tallHorP (Id X) (Id Y) = Unit (tallMerge X Y).E := by
2610     exact CategoryTheory.Functor.ext (F := tallHorP (Id X) (Id Y))
2611     (G := Unit (tallMerge X Y).E) (fun x => by
2612       rcases x with <|<n|>, v|>
2613       cases n with
2614       | zero => rfl
2615       | succ n => cases v <,> rfl)
2616   have hA : HEq (tallHorMap (Id X) (Id Y)).A := by
2617     (Id (tallMerge X Y) : TallAtlasHom _ _).A := by
2618     apply NatTrans.hex_right _ _
2619     (congrArg (fun P => P then (tallMerge X Y).G) hP)
2620     intro x
2621     rcases x with <|<n|>, v|>

```

```

2622 cases n with
2623 | zero =>
2624   apply heq_of_eq
2625   apply DomIns.hom_ext
2626   intro z
2627   apply Subtype.ext
2628   simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap]
2629   rw [TallAtlas.extentMap_id, TallAtlas.extentMap_id]
2630   change Sum.map (Id X.extent) (Id Y.extent) z.1 = z.1
2631   rcases z.1 with a | b <|> rfl
2632 | succ n =>
2633   rcases v with v | v <|>
2634   apply heq_of_eq <|>
2635   apply DomIns.hom_ext <|>
2636   intro z <|>
2637   apply Subtype.ext <|>
2638   simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap] <|>
2639   rw [TallAtlas.extentMap_id, TallAtlas.extentMap_id] <|>
2640   change Sum.map (Id X.extent) (Id Y.extent) z.1 = z.1 <|>
2641   rcases z.1 with a | b <|> rfl
2642 exact TallAtlasHom.ext _ _ hP hA
2643
2644 theorem tallHorMap_comp {X_1 X_2 Y_1 Y_2 Z_1 Z_2 : TallAtlas}
2645   (f_1 : X_1 --> Y_1) (f_2 : Y_1 --> Z_1) (g_1 : X_2 --> Y_2) (g_2 : Y_2 --> Z_2) :
2646   tallHorMap (f_1 >> f_2) (g_1 >> g_2) =
2647     tallHorMap f_1 g_1 >> tallHorMap f_2 g_2 := by
2648   have hP : tallHorP (f_1 >> f_2) (g_1 >> g_2) =
2649     tallHorP f_1 g_1 then tallHorP f_2 g_2 := by
2650     exact CategoryTheory.Functor.ext
2651       (F := tallHorP (f_1 >> f_2) (g_1 >> g_2))
2652       (G := tallHorP f_1 g_1 then tallHorP f_2 g_2) (fun x => by
2653         rcases x with <|<|n|>, v|>
2654         cases n with
2655         | zero => rfl
2656         | succ n => cases v <|> rfl)
2657   have hA : HEq (tallHorMap (f_1 >> f_2) (g_1 >> g_2)).A
2658     (tallHorMap f_1 g_1 >> tallHorMap f_2 g_2).A := by
2659     apply NatTrans.hext_right _ _
2660     (congrArg (fun P => P then (tallMerge Z_1 Z_2).G) hP)
2661   intro x
2662   rcases x with <|<|n|>, v|>
2663   cases n with
2664   | zero =>
2665     apply heq_of_eq
2666     apply DomIns.hom_ext
2667     intro z
2668     apply Subtype.ext
2669     simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap,
2670       TallAtlas.extentMap_comp]
2671     change (domSum.map
2672       (TallAtlas.extentMap f_1 >> TallAtlas.extentMap f_2)
2673       (TallAtlas.extentMap g_1 >> TallAtlas.extentMap g_2)) z.1 =
2674       domSum.map (TallAtlas.extentMap f_2) (TallAtlas.extentMap g_2)
2675       (domSum.map (TallAtlas.extentMap f_1) (TallAtlas.extentMap g_1) z.1)
2676     rcases z.1 with a | b <|> rfl
2677   | succ n =>
2678     rcases v with v | v <|>
2679     apply heq_of_eq <|>
2680     apply DomIns.hom_ext <|>
2681     intro z <|>
2682     apply Subtype.ext <|>
2683     simp only [tallHorMap, tallHorA, imageDom.mapAcross, tallHorExtentMap,
2684       TallAtlas.extentMap_comp] <|>
2685     change (domSum.map
2686       (TallAtlas.extentMap f_1 >> TallAtlas.extentMap f_2)
2687       (TallAtlas.extentMap g_1 >> TallAtlas.extentMap g_2)) z.1 =
2688       domSum.map (TallAtlas.extentMap f_2) (TallAtlas.extentMap g_2)
2689       (domSum.map (TallAtlas.extentMap f_1) (TallAtlas.extentMap g_1) z.1) <|>
2690     rcases z.1 with a | b <|> rfl
2691   exact TallAtlasHom.ext _ _ hP hA
2692
2693 /-- The horizontal sum on raw infinite-spine presentations. This is the
2694 implementation bifunctor from which the coherent atlas operation is derived. -/

```

```

2695 def TallAtlHorSum : TallAtlas * TallAtlas ==> TallAtlas where
2696   obj X := tallMerge X.1 X.2
2697   map f := tallHorMap f.1 f.2
2698   map_id X := tallHorMap_id X.1 X.2
2699   map_comp f g := tallHorMap_comp f.1 g.1 f.2 g.2
2700
2701 def domSum.swap (X Y : DomIns) : domSum X Y --> domSum Y X where
2702   toFun z := z.swap
2703   inj' := by
2704     intro a b h
2705     rcases a with a | a <|> rcases b with b | b
2706     . exact congrArg Sum.inl (Sum.inr.inj h)
2707     . cases h
2708     . cases h
2709     . exact congrArg Sum.inr (Sum.inl.inj h)
2710
2711 def tallSwapPObj (X Y : TallAtlas) (x : (tallMerge X Y).E) :
2712   (tallMerge Y X).E := by
2713     rcases x with <|<|n|>, v|>
2714     cases n with
2715     | zero => exact (tallMerge Y X).originElement
2716     | succ n =>
2717       exact match v with
2718       | .inl k => tallMergeRightObj Y X <|op n, k|>
2719       | .inr k => tallMergeLeftObj Y X <|op n, k|>
2720
2721 def tallSwapPHom (X Y : TallAtlas) {x y : (tallMerge X Y).E} (q : x --> y) :
2722   tallSwapPObj X Y x --> tallSwapPObj X Y y := by
2723     rcases x with <|<|mx|>, vx|>
2724     rcases y with <|<|my|>, vy|>
2725     cases mx with
2726     | zero =>
2727       have hmy : my = 0 := by
2728         have hq := leOfHom q.val.unop
2729         change my <= 0 at hq
2730         omega
2731       subst my
2732       exact Id _
2733     | succ n =>
2734       cases my with
2735       | zero => exact (tallMerge Y X).toOrigin _
2736       | succ p =>
2737         have hpn : p <= n := Nat.succ_le_succ_iff.mp (leOfHom q.val.unop)
2738         rcases vx with k | k
2739         . have hqv : Sum.inl (X.H.map (homOfLE hpn).op k) = vy := by
2740             simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2741           subst vy
2742           exact tallMergeRightMap Y X (CategoryOfElements.homMk _ _
2743             (homOfLE hpn).op rfl)
2744         . have hqv : Sum.inr (Y.H.map (homOfLE hpn).op k) = vy := by
2745             simpa [tallMerge, tallMergeH, tallMergePageMap] using q.property
2746           subst vy
2747           exact tallMergeLeftMap Y X (CategoryOfElements.homMk _ _
2748             (homOfLE hpn).op rfl)
2749
2750 def tallSwapP (X Y : TallAtlas) : (tallMerge X Y).E ==> (tallMerge Y X).E where
2751   obj := tallSwapPObj X Y
2752   map := tallSwapPHom X Y
2753   map_id _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2754   map_comp _ _ := by apply CategoryOfElements.ext; apply Subsingleton.elim
2755
2756 theorem tallSwapRootFactor (X Y : TallAtlas) (x : (tallMerge X Y).E)
2757   (a : tallMergeRawDom X Y x) :
2758   exists b : tallMergeRawDom Y X (tallSwapPObj X Y x),
2759     domSum.swap X.extent Y.extent (tallMergeRootMap X Y x a) =
2760     tallMergeRootMap Y X (tallSwapPObj X Y x) b := by
2761     rcases x with <|<|n|>, v|>
2762     cases n with
2763     | zero => exact <|a.swap, rfl|>
2764     | succ n =>
2765       rcases v with k | k
2766       . exact <|a, rfl|>
2767       . exact <|a, rfl|>

```

```

2768
2769 def TallAtIBrd (X Y : TallAtlas) : tallMerge X Y --> tallMerge Y X where
2770   P := tallSwapP X Y
2771   A :=
2772     { app := fun x => imageDom.mapAcross
2773       (tallMergeRootMap X Y x)
2774       (tallMergeRootMap Y X (tallSwapPObj X Y x))
2775       (domSum.swap X.extent Y.extent) (tallSwapRootFactor X Y x)
2776     naturality := by
2777       intro x y q
2778       apply DomIns.hom_ext
2779       intro z
2780       apply Subtype.ext
2781       rfl }
2782
2783 theorem TallAtIBrd_involutive (X Y : TallAtlas) :
2784   TallAtIBrd X Y >> TallAtIBrd Y X = Id (tallMerge X Y) := by
2785   have hP : tallSwapP X Y then tallSwapP Y X = Unit (tallMerge X Y).E := by
2786     exact CategoryTheory.Functor.ext (fun x => by
2787       rcases x with <|<n|>, v|>
2788       cases n with
2789       | zero => rfl
2790       | succ n => cases v <|> rfl)
2791   apply TallAtlasHom.ext _ _ hP
2792   apply NatTrans.hext_right _ _
2793   (congrArg (fun P => P then (tallMerge X Y).G) hP)
2794   intro x
2795   rcases x with <|<n|>, v|>
2796   cases n with
2797   | zero =>
2798     apply heq_of_eq
2799     apply DomIns.hom_ext
2800     intro z
2801     apply Subtype.ext
2802     simp only [TallAtIBrd, imageDom.mapAcross]
2803     change domSum.swap Y.extent X.extent
2804     (domSum.swap X.extent Y.extent z.1) = z.1
2805     rcases z.1 with a | b <|> rfl
2806   | succ n =>
2807     rcases v with v | v <|>
2808     apply heq_of_eq <|>
2809     apply DomIns.hom_ext <|>
2810     intro z <|>
2811     apply Subtype.ext <|>
2812     simp only [TallAtIBrd, imageDom.mapAcross] <|>
2813     change domSum.swap Y.extent X.extent
2814     (domSum.swap X.extent Y.extent z.1) = z.1 <|>
2815     rcases z.1 with a | b <|> rfl
2816
2817 def TallAtIBrdIso (X Y : TallAtlas) : tallMerge X Y Iso tallMerge Y X where
2818   hom := TallAtIBrd X Y
2819   inv := TallAtIBrd Y X
2820   hom_inv_id := TallAtIBrd_involutive X Y
2821   inv_hom_id := TallAtIBrd_involutive Y X
2822
2823 /-- The empty atlas regarded on its full spine. -/
2824 def TallAtlasI : TallAtlas := AtII.tall
2825
2826 theorem domIns_eqToHom_apply {A B : DomIns} (h : A = B) (a : A) :
2827   (eqToHom h : A --> B) a = h transport a := by
2828     cases h
2829     rfl
2830
2831 theorem castDep_symm_cast {A : Sort u} (P : A -> Sort*) {x y : A}
2832   (h : x = y) (a : P y) : h transport (h.symm transport a) = a := by
2833     cases h
2834     rfl
2835
2836 def bouquetRawDomIndex (X Y : Atl) :
2837   (m : Fin (bouquetLength X.P.folio Y.P.folio)) ->
2838   (bouquetFolio X.P.folio Y.P.folio).H.obj (op m) -> DomIns :=
2839   Fin.cases (fun _ => domSum (extent X) (extent Y)) (fun n v =>
2840     match ofLex v with

```

```

2841 | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
2842 | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))
2843
2844 theorem bouquetLeftRawDom_eq (X Y : At1) (x : X.E) :
2845   bouquetRawDomIndex X Y (bouquetLeftElement X Y x).1.unop
2846   (bouquetLeftElement X Y x).2 = X.G.obj x := by
2847   rcases x with <|<|m|>, k|>
2848   simp only [bouquetLeftElement, bouquetLeftPred, bouquetRawDomIndex,
2849     bouquetChain, Fin.cases_succ]
2850   split
2851   . rename_i k' hk
2852     change Sum.inl _ = Sum.inl k' at hk
2853     have hk' := Sum.inl.inj hk
2854     subst k'
2855     congr 1
2856     apply Functor.Elements.ext (F := X.P.H) _ _
2857       (congrArg op (X.P.folio.paddedIndex_fin m))
2858     dsimp only [Pag.cell]
2859     change X.P.H.map (eqToHom _) (X.P.H.map (eqToHom _) k) = k
2860     rw [<- FunctorToTypes.map_comp_apply]
2861     simp
2862   . rename_i k' hk
2863     change Sum.inl _ = Sum.inr k' at hk
2864     cases hk
2865
2866 theorem bouquetRightRawDom_eq (X Y : At1) (y : Y.E) :
2867   bouquetRawDomIndex X Y (bouquetRightElement X Y y).1.unop
2868   (bouquetRightElement X Y y).2 = Y.G.obj y := by
2869   rcases y with <|<|m|>, k|>
2870   simp only [bouquetRightElement, bouquetRightPred, bouquetRawDomIndex,
2871     bouquetChain, Fin.cases_succ]
2872   split
2873   . rename_i k' hk
2874     change Sum.inr _ = Sum.inl k' at hk
2875     cases hk
2876   . rename_i k' hk
2877     change Sum.inr _ = Sum.inr k' at hk
2878     have hk' := Sum.inr.inj hk
2879     subst k'
2880     congr 1
2881     apply Functor.Elements.ext (F := Y.P.H) _ _
2882       (congrArg op (Y.P.folio.paddedIndex_fin m))
2883     dsimp only [Pag.cell]
2884     change Y.P.H.map (eqToHom _) (Y.P.H.map (eqToHom _) k) = k
2885     rw [<- FunctorToTypes.map_comp_apply]
2886     simp
2887
2888 def paddedElement (X : At1) (x : X.E) : X.E :=
2889   X.P.cell (op (X.P.folio.paddedIndex x.1.unop.1))
2890   (X.P.H.map
2891     (eqToHom (congrArg op (X.P.folio.paddedIndex_fin x.1.unop).symm)) x.2)
2892
2893 theorem paddedElement_eq (X : At1) (x : X.E) : paddedElement X x = x := by
2894   apply Functor.Elements.ext (F := X.P.H) _ _
2895     (congrArg op (X.P.folio.paddedIndex_fin x.1.unop))
2896   dsimp only [paddedElement, Pag.cell]
2897   change X.P.H.map (eqToHom _) (X.P.H.map (eqToHom _) x.2) = x.2
2898   rw [<- FunctorToTypes.map_comp_apply]
2899   simp
2900
2901 def bouquetRootIndex (X Y : At1) :
2902   forall (m : Fin (bouquetLength X.P.folio Y.P.folio))
2903     (v : (bouquetFolio X.P.folio Y.P.folio).H.obj (op m)),
2904     bouquetRawDomIndex X Y m v --> domSum (extent X) (extent Y) := by
2905   intro m
2906   induction m using Fin.cases with
2907   | zero => intro v; exact Id _
2908   | succ n =>
2909     intro v
2910     change (match ofLex v with
2911       | .inl k => X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
2912       | .inr k => Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k))
2913       --> domSum (extent X) (extent Y)

```

```

2914   exact match ofLex v with
2915     | .inl k => X.G.map (X.P.cellToOrigin
2916       (op (X.P.folio.paddedIndex n.1)) k) >>
2917       domSum.inl (extent X) (extent Y)
2918     | .inr k => Y.G.map (Y.P.cellToOrigin
2919       (op (Y.P.folio.paddedIndex n.1)) k) >>
2920       domSum.inr (extent X) (extent Y)
2921
2922 @[simp]
2923 theorem bouquetRootIndex_succ_left (X Y : Atl)
2924   (n : Fin (bouquetDepth X.P.folio Y.P.folio))
2925   (k : X.P.H.obj (op (X.P.folio.paddedIndex n.1)))
2926   (a : X.G.obj (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)) :
2927   bouquetRootIndex X Y n.succ (toLex (Sum.inl k)) a =
2928     Sum.inl (X.G.map
2929       (X.P.cellToOrigin (op (X.P.folio.paddedIndex n.1)) k) a) := by
2930   rfl
2931
2932 @[simp]
2933 theorem bouquetRootIndex_succ_right (X Y : Atl)
2934   (n : Fin (bouquetDepth X.P.folio Y.P.folio))
2935   (k : Y.P.H.obj (op (Y.P.folio.paddedIndex n.1)))
2936   (a : Y.G.obj (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)) :
2937   bouquetRootIndex X Y n.succ (toLex (Sum.inr k)) a =
2938     Sum.inr (Y.G.map
2939       (Y.P.cellToOrigin (op (Y.P.folio.paddedIndex n.1)) k) a) := by
2940   rfl
2941
2942 theorem bouquetRootIndex_left (X Y : Atl) (x : X.E)
2943   (a : bouquetRawDomIndex X Y (bouquetLeftElement X Y x).1.unop
2944     (bouquetLeftElement X Y x).2) :
2945   bouquetRootIndex X Y (bouquetLeftElement X Y x).1.unop
2946     (bouquetLeftElement X Y x).2 a =
2947     Sum.inl (originImage X x (bouquetLeftRawDom_eq X Y x transport a)) := by
2948   rcases x with <|<|m|>, k|>
2949   change bouquetRootIndex X Y (bouquetLeftPred X Y m).succ
2950     (toLex (Sum.inl (X.P.H.map
2951       (eqToHom (congrArg op (X.P.folio.paddedIndex_fin m).symm)) k))) a = _
2952   rw [bouquetRootIndex_succ_left]
2953   congr 1
2954   change X.G.map (X.P.folio.toOrigin (paddedElement X <|op m, k|>)) a =
2955     X.G.map (X.P.folio.toOrigin <|op m, k|>)
2956       (bouquetLeftRawDom_eq X Y <|op m, k|> transport a)
2957   let q := paddedElement_eq X (<|op m, k|> : X.E)
2958   have hto : eqToHom q >> X.P.folio.toOrigin <|op m, k|> =
2959     X.P.folio.toOrigin (paddedElement X <|op m, k|>) := by
2960     apply CategoryOfElements.ext
2961     apply Subsingleton.elim
2962   rw [<- hto, X.G.map_comp]
2963   change X.G.map (X.P.folio.toOrigin <|op m, k|>)
2964     (X.G.map (eqToHom q) a) =
2965     X.G.map (X.P.folio.toOrigin <|op m, k|>)
2966       (bouquetLeftRawDom_eq X Y <|op m, k|> transport a)
2967   apply congrArg (fun z => X.G.map (X.P.folio.toOrigin <|op m, k|>) z)
2968   have hmap := eqToHom_map X.G q
2969   rw [hmap]
2970   have heq : congrArg X.G.obj q = bouquetLeftRawDom_eq X Y <|op m, k|> :=
2971     Subsingleton.elim _ _
2972   cases heq
2973   exact domIns_eqToHom_apply _ _
2974
2975 theorem bouquetRootIndex_right (X Y : Atl) (y : Y.E)
2976   (a : bouquetRawDomIndex X Y (bouquetRightElement X Y y).1.unop
2977     (bouquetRightElement X Y y).2) :
2978   bouquetRootIndex X Y (bouquetRightElement X Y y).1.unop
2979     (bouquetRightElement X Y y).2 a =
2980     Sum.inr (originImage Y y (bouquetRightRawDom_eq X Y y transport a)) := by
2981   rcases y with <|<|m|>, k|>
2982   change bouquetRootIndex X Y (bouquetRightPred X Y m).succ
2983     (toLex (Sum.inr (Y.P.H.map
2984       (eqToHom (congrArg op (Y.P.folio.paddedIndex_fin m).symm)) k))) a = _
2985   rw [bouquetRootIndex_succ_right]
2986   congr 1

```



```

2987 change Y.G.map (Y.P.folio.toOrigin (paddedElement Y <|op m, k|>)) a =
2988   Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
2989     (bouquetRightRawDom_eq X Y <|op m, k|> transport a)
2990 let q := paddedElement_eq Y (<|op m, k|> : Y.E)
2991 have hto : eqToHom q >> Y.P.folio.toOrigin <|op m, k|> =
2992   Y.P.folio.toOrigin (paddedElement Y <|op m, k|>) := by
2993     apply CategoryOfElements.ext
2994     apply Subsingleton.elim
2995 rw [<- hto, Y.G.map_comp]
2996 change Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
2997   (Y.G.map (eqToHom q) a) =
2998   Y.G.map (Y.P.folio.toOrigin <|op m, k|>)
2999     (bouquetRightRawDom_eq X Y <|op m, k|> transport a)
3000 apply congrArg (fun z => Y.G.map (Y.P.folio.toOrigin <|op m, k|>) z)
3001 have hmap := eqToHom_map Y.G q
3002 rw [hmap]
3003 have heq : congrArg Y.G.obj q = bouquetRightRawDom_eq X Y <|op m, k|> :=
3004   Subsingleton.elim _ _
3005 cases heq
3006 exact domIns_eqToHom_apply _ _
3007
3008 def bouquetDom (X Y : At1) (x : (bouquetPag X Y).E) : DomIns :=
3009   imageDom (bouquetRootIndex X Y x.1.unop x.2)
3010
3011 theorem bouquetRange_mono (X Y : At1) {x y : (bouquetPag X Y).E}
3012   (f : x --> y) :
3013     Set.range (bouquetRootIndex X Y x.1.unop x.2) subset
3014       Set.range (bouquetRootIndex X Y y.1.unop y.2) := by
3015   intro z hz
3016   rcases x with <|<|mx|>, vx|>
3017   rcases y with <|<|my|>, vy|>
3018   rcases hz with <|a, ha|>
3019   rw [<- ha]
3020   induction mx using Fin.cases with
3021   | zero =>
3022     induction my using Fin.cases with
3023     | zero => exact <|a, rfl|>
3024     | succ p =>
3025       have hf : p.succ -->
3026         (<|0, bouquetLength_pos X.P.folio Y.P.folio|> :
3027           Fin (bouquetLength X.P.folio Y.P.folio)) :=
3028         Quiver.Hom.unop f.val
3029       have h := leOfHom hf
3030       change p.1 + 1 <= 0 at h
3031       omega
3032   | succ n =>
3033     induction my using Fin.cases with
3034     | zero => exact <|bouquetRootIndex X Y n.succ vx a, rfl|>
3035     | succ p =>
3036       have hf : p.succ --> n.succ := Quiver.Hom.unop f.val
3037       have hpn : p <= n := Fin.succ_le_succ_iff.mp (leOfHom hf)
3038       let hxp : X.P.folio.paddedIndex p.1 <= X.P.folio.paddedIndex n.1 :=
3039         X.P.folio.paddedIndex_mono hpn
3040       let hyp : Y.P.folio.paddedIndex p.1 <= Y.P.folio.paddedIndex n.1 :=
3041         Y.P.folio.paddedIndex_mono hpn
3042       generalize hvx : ofLex vx = s
3043       rcases s with k | k
3044       . have evx : vx = toLex (Sum.inl k) :=
3045         ofLex.injective (by simp using hvx)
3046       subst vx
3047       let k' := X.P.H.map (homOfLE hxp).op k
3048       have ef : f.val = (homOfLE (show p.succ <= n.succ from
3049         Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
3050       have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inl k)) = vy :=
3051         f.property
3052       have hvy : ofLex vy = Sum.inl k' := by
3053         rw [<- hfv, ef]
3054         change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inl k))))) = _
3055         simp [k']
3056         congr 2
3057       have evy : vy = toLex (Sum.inl k') :=
3058         ofLex.injective (by simp using hvy)
3059       subst vy

```



```

3060   let q := CategoryOfElements.homMk
3061     (X.P.cell (op (X.P.folio.paddedIndex n.1)) k)
3062     (X.P.cell (op (X.P.folio.paddedIndex p.1)) k')
3063     (homOfLE hxp).op rfl
3064   refine <|X.G.map q a, ?_|>
3065   change Sum.inl (X.G.map (X.P.cellToOrigin _ k') (X.G.map q a)) =
3066     Sum.inl (X.G.map (X.P.cellToOrigin _ k) a)
3067   apply congrArg Sum.inl
3068   have hc : q >> X.P.cellToOrigin _ k' = X.P.cellToOrigin _ k :=
3069     CategoryOfElements.ext X.P.H _ _ (Subsingleton.elim _ _)
3070   have hm := X.G.map_comp q (X.P.cellToOrigin _ k')
3071   rw [hc] at hm
3072   exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
3073   · have evx : vx = toLex (Sum.inr k) :=
3074     ofLex.injective (by simp using hvx)
3075   subst vx
3076   let k' := Y.P.H.map (homOfLE hyp).op k
3077   have ef : f.val = (homOfLE (show p.succ <= n.succ from
3078     Fin.succ_le_succ_iff.mpr hpn)).op := Subsingleton.elim _ _
3079   have hfv : (bouquetPag X Y).H.map f.val (toLex (Sum.inr k)) = vy :=
3080     f.property
3081   have hvy : ofLex vy = Sum.inr k' := by
3082     rw [<- hfv, ef]
3083     change ofLex (toLex (Sum.map _ _ (ofLex (toLex (Sum.inr k))))) = _
3084     simp [k']
3085     congr 2
3086   have evy : vy = toLex (Sum.inr k') :=
3087     ofLex.injective (by simp using hvy)
3088   subst vy
3089   let q := CategoryOfElements.homMk
3090     (Y.P.cell (op (Y.P.folio.paddedIndex n.1)) k)
3091     (Y.P.cell (op (Y.P.folio.paddedIndex p.1)) k')
3092     (homOfLE hyp).op rfl
3093   refine <|Y.G.map q a, ?_|>
3094   change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k') (Y.G.map q a)) =
3095     Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a)
3096   apply congrArg Sum.inr
3097   have hc : q >> Y.P.cellToOrigin _ k' = Y.P.cellToOrigin _ k :=
3098     CategoryOfElements.ext Y.P.H _ _ (Subsingleton.elim _ _)
3099   have hm := Y.G.map_comp q (Y.P.cellToOrigin _ k')
3100   rw [hc] at hm
3101   exact congrFun (congrArg Function.Embedding.toFun hm.symm) a
3102
3103 @[simp]
3104 theorem imageDom.map_val {A B R : DomIns} (f : A --> R) (g : B --> R)
3105   (h : Set.range f subset Set.range g) (z : imageDom f) :
3106   (imageDom.map f g h z).1 = z.1 := rfl
3107
3108 def bouquetImageMap (X Y : Atl) {x y : (bouquetPag X Y).E} (f : x --> y) :
3109   bouquetDom X Y x --> bouquetDom X Y y :=
3110   imageDom.map _ _ (bouquetRange_mono X Y f)
3111
3112 def bouquetFunctor (X Y : Atl) : (bouquetPag X Y).E ==> DomIns where
3113   obj := bouquetDom X Y
3114   map := bouquetImageMap X Y
3115   map_id := by
3116     intro x
3117     apply DomIns.hom_ext
3118     intro z
3119     apply Subtype.ext
3120     change z.1 = z.1
3121     rfl
3122   map_comp := by
3123     intro x y z f g
3124     apply DomIns.hom_ext
3125     intro w
3126     apply Subtype.ext
3127     change w.1 = w.1
3128     rfl
3129
3130 def bouquetAtlas (X Y : Atl) : Atl where
3131   P := bouquetPag X Y
3132   G := bouquetFunctor X Y

```

```

3133 disjoint := by
3134   intro m i j hij x y hxy
3135   rcases m with <|m|>
3136   induction m using Fin.cases with
3137   | zero =>
3138     exact hij ((bouquetFolio X.P.folio Y.P.folio).originEquiv.injective
3139       (Subsingleton.elim _ _))
3140   | succ n =>
3141     rcases x.2 with <|a, ha|>
3142     rcases y.2 with <|b, hb|>
3143     have huv : x.1 = y.1 := by
3144       have e := congrArg (fun q => q.1) hxy
3145       change x.1 = y.1 at e
3146       exact e
3147     have hroot : bouquetRootIndex X Y n.succ i a =
3148       bouquetRootIndex X Y n.succ j b := ha.trans (huv.trans hb.symm)
3149     generalize hi : ofLex i = si
3150     generalize hj : ofLex j = sj
3151     rcases si with k | k <;> rcases sj with l | l
3152     . have ei : i = toLex (Sum.inl k) :=
3153       ofLex.injective (by simp using hi)
3154     have ej : j = toLex (Sum.inl l) :=
3155       ofLex.injective (by simp using hj)
3156     subst i
3157     subst j
3158     have hkl : k != l := by
3159       intro e
3160       subst l
3161       exact hij rfl
3162     change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
3163       Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
3164     exact X.disjoint _ k l hkl a b (Sum.inl.inj hroot)
3165     . have ei : i = toLex (Sum.inl k) :=
3166       ofLex.injective (by simp using hi)
3167     have ej : j = toLex (Sum.inr l) :=
3168       ofLex.injective (by simp using hj)
3169     subst i
3170     subst j
3171     change Sum.inl (X.G.map (X.P.cellToOrigin _ k) a) =
3172       Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
3173     exact Sum.noConfusion hroot
3174     . have ei : i = toLex (Sum.inr k) :=
3175       ofLex.injective (by simp using hi)
3176     have ej : j = toLex (Sum.inl l) :=
3177       ofLex.injective (by simp using hj)
3178     subst i
3179     subst j
3180     change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
3181       Sum.inl (X.G.map (X.P.cellToOrigin _ l) b) at hroot
3182     exact Sum.noConfusion hroot
3183     . have ei : i = toLex (Sum.inr k) :=
3184       ofLex.injective (by simp using hi)
3185     have ej : j = toLex (Sum.inr l) :=
3186       ofLex.injective (by simp using hj)
3187     subst i
3188     subst j
3189     have hkl : k != l := by
3190       intro e
3191       subst l
3192       exact hij rfl
3193     change Sum.inr (Y.G.map (Y.P.cellToOrigin _ k) a) =
3194       Sum.inr (Y.G.map (Y.P.cellToOrigin _ l) b) at hroot
3195     exact Y.disjoint _ k l hkl a b (Sum.inr.inj hroot)
3196
3197 /-- The object-level atlas merge used by the horizontal construction. -/
3198 def AtlMerge (X Y : Atl) : Atl := bouquetAtlas X Y
3199
3200 def bouquetTallPred (X Y : Atl) (n : Nat) :
3201   Fin (bouquetDepth X.P.folio Y.P.folio) :=
3202   <|min n (bouquetDepth X.P.folio Y.P.folio - 1), by
3203     have hp : 0 < bouquetDepth X.P.folio Y.P.folio :=
3204       lt_of_lt_of_le X.P.folio.positive (le_max_left _ _)
3205     omega|>

```

```

3206
3207 theorem bouquet_padded_succ (X Y : Atl) (n : Nat) :
3208   (bouquetFolio X.P.folio Y.P.folio).paddedIndex (n + 1) =
3209   (bouquetTallPred X Y n).succ := by
3210   apply Fin.ext
3211   simp only [Folio.paddedIndex, bouquetFolio, bouquetLength, bouquetTallPred,
3212     bouquetDepth, Fin.val_succ]
3213   have hp : 0 < max X.P.folio.length Y.P.folio.length :=
3214     lt_of_lt_of_le X.P.folio.positive (le_max_left _ _)
3215   omega
3216
3217 theorem left_padded_tallPred (X Y : Atl) (n : Nat) :
3218   X.P.folio.paddedIndex (bouquetTallPred X Y n).1 =
3219   X.P.folio.paddedIndex n := by
3220   apply Fin.ext
3221   simp only [Folio.paddedIndex, bouquetTallPred, bouquetDepth]
3222   have hx : X.P.folio.length <= max X.P.folio.length Y.P.folio.length :=
3223     le_max_left _ _
3224   omega
3225
3226 theorem right_padded_tallPred (X Y : Atl) (n : Nat) :
3227   Y.P.folio.paddedIndex (bouquetTallPred X Y n).1 =
3228   Y.P.folio.paddedIndex n := by
3229   apply Fin.ext
3230   simp only [Folio.paddedIndex, bouquetTallPred, bouquetDepth]
3231   have hy : Y.P.folio.length <= max X.P.folio.length Y.P.folio.length :=
3232     le_max_right _ _
3233   omega
3234
3235 /-- The finite coherent implementation of `AtlMerge` presents exactly the
3236 componentwise positive pages used by `tallMerge`. -/
3237 def AtlMerge.pageEquiv (X Y : Atl) (n : Nat) :
3238   (AtlMerge X Y).page (n + 1) Equiv Sum (X.page n) (Y.page n) := by
3239   change (bouquetChain X.P.folio Y.P.folio
3240     ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (n + 1))).Obj Equiv _
3241   rw [bouquet_padded_succ]
3242   change Lex (Sum
3243     ((X.P.folio.core.obj (X.P.folio.paddedIndex
3244       (bouquetTallPred X Y n).1)).unop.Obj)
3245     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
3246       (bouquetTallPred X Y n).1)).unop.Obj)) Equiv _
3247   rw [left_padded_tallPred, right_padded_tallPred]
3248   exact ofLex
3249
3250 /-- The left input as a page-preserving subobject of the tall presentation of
3251 an atlas merge. In particular, occurrence `n` is sent to occurrence `n+1`;
3252 no finite representative is selected here. -/
3253 def tallBouquetLeftElement (X Y : Atl) (x : X.tall.E) : (AtlMerge X Y).tall.E := by
3254   refine <|op (x.1.unop + 1), ?_|>
3255   change (bouquetChain X.P.folio Y.P.folio
3256     ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (x.1.unop + 1))).Obj
3257   rw [bouquet_padded_succ]
3258   change Lex (Sum
3259     ((X.P.folio.core.obj (X.P.folio.paddedIndex
3260       (bouquetTallPred X Y x.1.unop).1)).unop.Obj)
3261     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
3262       (bouquetTallPred X Y x.1.unop).1)).unop.Obj))
3263   rw [left_padded_tallPred]
3264   exact toLex (Sum.inl x.2)
3265
3266 /-- The right input in the tall presentation of an atlas merge. -/
3267 def tallBouquetRightElement (X Y : Atl) (y : Y.tall.E) : (AtlMerge X Y).tall.E := by
3268   refine <|op (y.1.unop + 1), ?_|>
3269   change (bouquetChain X.P.folio Y.P.folio
3270     ((bouquetFolio X.P.folio Y.P.folio).paddedIndex (y.1.unop + 1))).Obj
3271   rw [bouquet_padded_succ]
3272   change Lex (Sum
3273     ((X.P.folio.core.obj (X.P.folio.paddedIndex
3274       (bouquetTallPred X Y y.1.unop).1)).unop.Obj)
3275     ((Y.P.folio.core.obj (Y.P.folio.paddedIndex
3276       (bouquetTallPred X Y y.1.unop).1)).unop.Obj))
3277   rw [right_padded_tallPred]
3278   exact toLex (Sum.inr y.2)

```

```

3279
3280 /-- The category of Atlas Federations used for iterated stable atlas merging. -/
3281 abbrev AtlFed := StableAtlasFamily
3282
3283 /-- The horizontal bifunctor is federation merge on objects and applies a
3284 stable traversal independently to every tagged component on arrows. -/
3285 def AtlHorSum : AtlFed * AtlFed ==> AtlFed := StableAtlHorSum
3286
3287 def AtlBrd (X Y : AtlFed) :
3288   AtlHorSum.obj (X, Y) Iso AtlHorSum.obj (Y, X) :=
3289   stableAtlasBraiding X Y
3290
3291 def AtlAsoc (X Y Z : AtlFed) :
3292   AtlHorSum.obj (AtlHorSum.obj (X, Y), Z) Iso
3293   AtlHorSum.obj (X, AtlHorSum.obj (Y, Z)) :=
3294   stableAtlasAssociator X Y Z
3295
3296 def AtlLu (X : AtlFed) : AtlHorSum.obj (stableAtlasUnit, X) Iso X :=
3297   stableAtlasLeftUnitor X
3298
3299 def AtlRu (X : AtlFed) : AtlHorSum.obj (X, stableAtlasUnit) Iso X :=
3300   stableAtlasRightUnitor X
3301
3302 theorem horizontalLemma : Nonempty (SymmetricCategory AtlFed) :=
3303   <|inferInstance|>
3304
3305 instance : SymmetricCategory StableAtlasFamilyop where
3306   symmetry X Y := by
3307     apply Quiver.Hom.unop_inj
3308     simp
3309
3310 theorem daTratInc_essImage (F : DaTratPresheaf) :
3311   DaTratInc.essImage F :=
3312   <|<|F, trivial|>, <|Iso.refl _|>|>
3313
3314 instance daTratPreservesTensorRightForTensor (v : Type 3)
3315   (d : StableAtlasFamilyop) :
3316   PreservesColimitsOfShape
3317   (CostructuredArrow (MonoidalCategory.tensor StableAtlasFamilyop) d)
3318   (MonoidalCategory.tensorRight v) :=
3319   preservesColimitsOfShape_of_natIso
3320   (BraidedCategory.tensorLeftIsoTensorRight v)
3321
3322 instance daTratPreservesTensorRightForUnit (v : Type 3)
3323   (d : StableAtlasFamilyop) :
3324   PreservesColimitsOfShape
3325   (CostructuredArrow
3326     (Functor.fromPUnit
3327       (MonoidalCategory.tensorUnit StableAtlasFamilyop)) d)
3328   (MonoidalCategory.tensorRight v) :=
3329   preservesColimitsOfShape_of_natIso
3330   (BraidedCategory.tensorLeftIsoTensorRight v)
3331
3332 instance daTratPreservesTensorRightForUnitProduct (v : Type 3)
3333   (d : StableAtlasFamilyop * StableAtlasFamilyop) :
3334   PreservesColimitsOfShape
3335   (CostructuredArrow
3336     ((Functor.id StableAtlasFamilyop).prod
3337       (Functor.fromPUnit
3338         (MonoidalCategory.tensorUnit StableAtlasFamilyop)))) d)
3339   (MonoidalCategory.tensorRight v) :=
3340   preservesColimitsOfShape_of_natIso
3341   (BraidedCategory.tensorLeftIsoTensorRight v)
3342
3343 instance daTratPreservesTensorRightForTensorProduct (v : Type 3)
3344   (d : StableAtlasFamilyop * StableAtlasFamilyop) :
3345   PreservesColimitsOfShape
3346   (CostructuredArrow
3347     ((MonoidalCategory.tensor StableAtlasFamilyop).prod
3348       (Functor.id StableAtlasFamilyop)) d)
3349   (MonoidalCategory.tensorRight v) :=
3350   preservesColimitsOfShape_of_natIso
3351   (BraidedCategory.tensorLeftIsoTensorRight v)

```

```

3352
3353 noncomputable def daTratMonoidal : MonoidalCategory DaTrat :=
3354   MonoidalCategory.monoidalOfHasDayConvolutions DaTratInc
3355   (ObjectProperty.fullyFaithfuliota IsStableDataTransformation)
3356   (fun _ _ => daTratInc_essImage _)
3357   (daTratInc_essImage _)
3358
3359 noncomputable instance : MonoidalCategory DaTrat := daTratMonoidal
3360
3361 noncomputable instance daTratLawful :
3362   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct
3363   StableAtlasFamilyop (Type 3) DaTrat :=
3364   MonoidalCategory.lawfulDayConvolutionMonoidalCategoryStructOfHasDayConvolutions
3365   DaTratInc (ObjectProperty.fullyFaithfuliota IsStableDataTransformation)
3366   (fun _ _ => daTratInc_essImage _)
3367   (daTratInc_essImage _)
3368
3369 noncomputable instance daTratDayConvolution (F G : DaTrat) :
3370   MonoidalCategory.DayConvolution (DaTratInc.obj F) (DaTratInc.obj G) :=
3371   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution
3372   StableAtlasFamilyop (Type 3) DaTrat F G
3373
3374 noncomputable instance daTratDayConvolutionRightNested (F G H : DaTrat) :
3375   MonoidalCategory.DayConvolution (DaTratInc.obj F)
3376   (MonoidalCategory.DayConvolution.convolution
3377    (DaTratInc.obj G) (DaTratInc.obj H)) :=
3378   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2
3379   StableAtlasFamilyop (Type 3) DaTrat F G H
3380
3381 noncomputable instance daTratDayConvolutionLeftNested (F G H : DaTrat) :
3382   MonoidalCategory.DayConvolution
3383   (MonoidalCategory.DayConvolution.convolution
3384    (DaTratInc.obj F) (DaTratInc.obj G))
3385   (DaTratInc.obj H) :=
3386   MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolution_2'
3387   StableAtlasFamilyop (Type 3) DaTrat F G H
3388
3389 noncomputable def daTratBraiding (F G : DaTrat) :
3390   MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F := by
3391   exact (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).preimageIso
3392   (MonoidalCategory.DayConvolution.braiding
3393    (DaTratInc.obj F) (DaTratInc.obj G))
3394
3395 theorem daTratInc_map_braiding_hom (F G : DaTrat) :
3396   DaTratInc.map (daTratBraiding F G).hom =
3397   (MonoidalCategory.DayConvolution.braiding
3398    (DaTratInc.obj F) (DaTratInc.obj G)).hom := by
3399   exact (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).map_preimage _
3400
3401 theorem daTratInc_map_tensorHom {F_1 F_2 G_1 G_2 : DaTrat}
3402   (f : F_1 --> F_2) (g : G_1 --> G_2) :
3403   DaTratInc.map (MonoidalCategory.tensorHom f g) =
3404   MonoidalCategory.DayConvolution.map
3405   (DaTratInc.map f) (DaTratInc.map g) := by
3406   simpa [DaTratInc, daTratLawful] using
3407   (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.iota_map_tensorHom_hom_eq_tensorHom
3408    StableAtlasFamilyop (Type 3) DaTrat f g)
3409
3410 theorem daTratInc_map_associator_hom (F G H : DaTrat) :
3411   DaTratInc.map (MonoidalCategory.associator F G H).hom =
3412   (MonoidalCategory.DayConvolution.associator
3413    (DaTratInc.obj F) (DaTratInc.obj G) (DaTratInc.obj H)).hom := by
3414   simpa [DaTratInc, daTratLawful] using
3415   (MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.
3416    iota_map_associator_hom_eq_associator_hom
3417    StableAtlasFamilyop (Type 3) DaTrat F G H)
3418
3419 noncomputable instance daTratBraided : BraidedCategory DaTrat where
3420   braiding := daTratBraiding
3421   braiding_naturality_right := fun X { _ _ } f => by
3422     rw [← MonoidalCategory.id_tensorHom, ← MonoidalCategory.tensorHom_id]
3423     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).map_injective
3424     simp only [Functor.map_comp, daTratInc_map_tensorHom,

```

```

3424     daTratInc_map_braiding_hom]
3425     exact MonoidalCategory.DayConvolution.braiding_naturality_right
3426     (DaTratInc.obj X) (DaTratInc.map f)
3427   braiding_naturality_left := fun {_ _} f Z => by
3428     rw [<- MonoidalCategory.tensorHom_id, <- MonoidalCategory.id_tensorHom]
3429     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).map_injective
3430     simp only [Functor.map_comp, daTratInc_map_tensorHom,
3431       daTratInc_map_braiding_hom]
3432     exact MonoidalCategory.DayConvolution.braiding_naturality_left
3433     (DaTratInc.map f) (DaTratInc.obj Z)
3434   hexagon_forward := fun X Y Z => by
3435     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).map_injective
3436     simp only [Functor.map_comp, daTratInc_map_associator_hom,
3437       daTratInc_map_braiding_hom]
3438     rw [<- MonoidalCategory.tensorHom_id, <- MonoidalCategory.id_tensorHom]
3439     simp only [daTratInc_map_tensorHom]
3440     exact MonoidalCategory.DayConvolution.hexagon_forward
3441     (DaTratInc.obj X) (DaTratInc.obj Y) (DaTratInc.obj Z)
3442   hexagon_reverse := fun X Y Z => by
3443     apply (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).map_injective
3444     simp only [Functor.map_comp, daTratInc_map_braiding_hom]
3445     rw [<- MonoidalCategory.id_tensorHom, <- MonoidalCategory.tensorHom_id]
3446     simp only [daTratInc_map_tensorHom]
3447     exact MonoidalCategory.DayConvolution.hexagon_reverse
3448     (DaTratInc.obj X) (DaTratInc.obj Y) (DaTratInc.obj Z)
3449
3450   set_option maxHeartbeats 2400000 in
3451   noncomputable instance : SymmetricCategory DaTrat where
3452     symmetry F G := by
3453       apply (ObjectProperty.fullyFaithfuliota IsStableDataTransformation).map_injective
3454       simp only [Functor.map_comp]
3455       exact MonoidalCategory.DayConvolution.symmetry
3456       (DaTratInc.obj F) (DaTratInc.obj G)
3457
3458   /-- `DaTratMon` is definitionally the category of stable data transformations equipped
3459   with the Day convolution instance above. -/
3460   abbrev DaTratMon := DaTrat
3461
3462   noncomputable def I : DaTratMon := MonoidalCategory.tensorUnit DaTratMon
3463
3464   noncomputable def I_dayConvolutionUnit :
3465     MonoidalCategory.DayConvolutionUnit (DaTratInc.obj I) :=
3466     MonoidalCategory.LawfulDayConvolutionMonoidalCategoryStruct.convolutionUnit
3467     StableAtlasFamilyop (Type 3) DaTratMon
3468
3469   noncomputable def HorSum : DaTratMon * DaTratMon ==> DaTratMon :=
3470     MonoidalCategory.tensor DaTratMon
3471
3472   noncomputable def Brd (F G : DaTratMon) :
3473     MonoidalCategory.tensorObj F G Iso MonoidalCategory.tensorObj G F :=
3474     daTratBraiding F G
3475
3476   noncomputable def Asoc (F G H : DaTratMon) :
3477     MonoidalCategory.tensorObj (MonoidalCategory.tensorObj F G) H Iso
3478     MonoidalCategory.tensorObj F (MonoidalCategory.tensorObj G H) :=
3479     MonoidalCategory.associator F G H
3480
3481   noncomputable def Lu (F : DaTratMon) :
3482     MonoidalCategory.tensorObj I F Iso F := MonoidalCategory.leftUnitor F
3483
3484   noncomputable def Ru (F : DaTratMon) :
3485     MonoidalCategory.tensorObj F I Iso F := MonoidalCategory.rightUnitor F
3486
3487   theorem serenityLemma : Nonempty (SymmetricCategory DaTratMon) :=
3488     <|inferInstance|>
3489   end
3490
3491   end Datra

```

Listing 1: Complete Lean formalization of DaTra